A Co-evolutionary Framework for Solving Binary Combinatorial Optimisation Problems

Submitted April 2026, in partial fulfilment of

the conditions for the award of the degree **BSc Hons Computer Science with Artificial Intelligence.**

Student's ID 20511811

School of Computer Science

University of Nottingham

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature _____LX_____

Date __15___/___April__/__2026___

Abstract

This dissertation investigates how probabilistic learning can be integrated directly into the operator-level decision-making of evolutionary algorithms on binary combinatorial optimisation problems. Traditional Memetic Algorithms rely on stochastic operators that make limited use of global structural information accumulated during search. To address this, a unified co-evolutionary framework is proposed that integrates a Population-Based Incremental Learning (PBIL) model with a Memetic Algorithm (MA), establishing a feedback loop in which a probability vector (PV) is learned from high-quality solutions and then used to guide local search and crossover at the bit level.

The key finding is that entropy-based guidance — directing operators toward bit positions where the model is least confident — consistently outperforms both unguided baselines and preference-based PBIL guidance. On MAX-SAT, the proposed PBIL-MA-Entropy framework won 11 of 12 benchmark instances and achieved an average rank of 1.167 across five competing algorithms under a shared evaluation budget, confirming that its advantage is not an artefact of additional computation. On the 0–1 Knapsack problem, cross-domain transfer was partial: entropy-guided local search remained competitive, and the full PBIL-MA framework performed comparably to the unguided MA-DBHC baseline (average ranks 3.458 versus 3.542), indicating that PBIL guidance provides negligible additional benefit on constrained problems where feasibility pressure already provides structural guidance to the search.

These results demonstrate that lightweight probabilistic models are most effective as operator-level guidance signals when they capture genuine structural uncertainty, rather than as standalone optimisers or preference-based directors. The framework's strength is domain-dependent: it delivers clear advantages where local search is the performance bottleneck, but adds less value on constrained problems where feasibility pressure already provides structural guidance.

Contents

1 Introduction

Combinatorial optimisation problems (COPs), such as MAX-SAT and the 0–1 Knapsack problem, are fundamental challenges in computer science. These problems are characterised by discrete decision spaces that grow exponentially with problem size, making exact methods impractical for many large instances [8]. As a result, metaheuristic approaches are widely used to obtain high-quality approximate solutions within reasonable time.

Among these approaches, evolutionary algorithms (EAs), including Genetic Algorithms [13] and Memetic Algorithms [24], have been widely adopted because of their robustness and flexibility. They evolve a population of candidate solutions through operators such as crossover, mutation, and local search. However, a key limitation of these methods is that operator decisions are often largely stochastic and make limited use of global knowledge accumulated during the search process.

In contrast, Estimation of Distribution Algorithms (EDAs) [17] provide a model-based approach by learning probabilistic information from promising solutions. A representative example is Population-Based Incremental Learning (PBIL) [2], which maintains a probability vector (PV) to capture the likelihood of decision variables taking particular values. This allows search to exploit global structural information, but PBIL is commonly used as a standalone model-based optimiser rather than as a direct guide for evolutionary operators.

This creates an important gap. Although both evolutionary algorithms and PBIL can perform well individually, learned probabilistic information is rarely integrated directly into the fine-grained decision-making of search operators, particularly local search and crossover. As a result, the potential of combining global probabilistic learning with operator-level search guidance remains underexplored.

To address this issue, this dissertation proposes a unified co-evolutionary framework that tightly integrates PBIL with a Memetic Algorithm. In the proposed framework, the PBIL probability vector is continuously updated from high-quality solutions in the evolving population and then used to guide key search operators. This establishes a feedback loop in which the probabilistic model learns from the evolutionary search process and subsequently influences it, enabling a more informed balance between exploration and exploitation.

Specifically, this dissertation pursues four objectives: (1) design and evaluate PBIL-guided local search strategies, particularly entropy-based bit-visitation ordering that prioritises positions where the model is least confident; (2) design and evaluate entropy-restricted uniform crossover, which uses PV confidence to gate recombination; (3) evaluate the full framework on MAX-SAT under both fixed-generation and shared-evaluation-budget conditions; and (4) test cross-domain generality by applying the same framework to the 0–1 Knapsack problem [22].

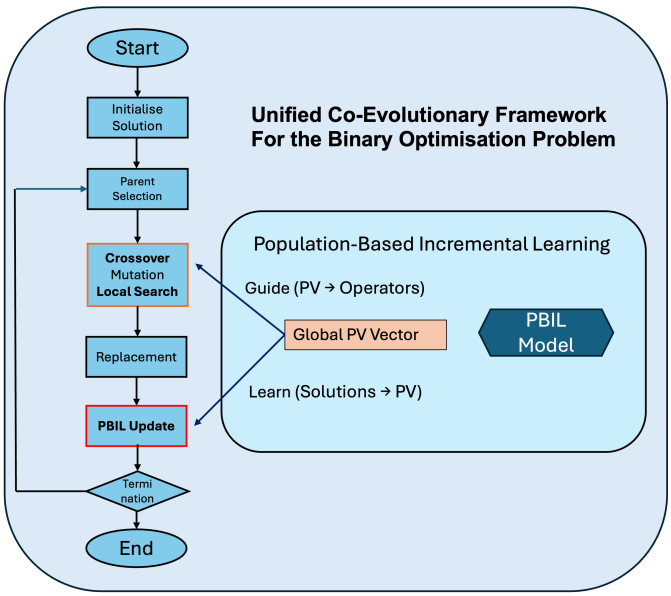


Figure 1: Overall pipeline of the unified co-evolutionary framework integrating PBIL with evolutionary search

2 Motivation

Evolutionary algorithms and memetic algorithms have demonstrated strong performance across a wide range of combinatorial optimisation problems due to their ability to balance exploration and exploitation [13, 24]. By maintaining a population of candidate solutions and iteratively applying variation and local improvement operators, these methods are capable of navigating complex and high-dimensional search spaces [24]. However, despite their success, a fundamental limitation remains: the search operators are largely uninformed and rely heavily on stochastic decisions [16].

In particular, operators such as crossover and local search typically select variables or solution components either randomly or based on simple heuristics [13, 16]. While local search improves exploitation by refining solutions, its behaviour is often determined by predefined rules that do not incorporate global information accumulated during the search process [16]. As a result, these operators may repeatedly explore unpromising regions or fail to prioritise the most impactful modifications.

At the same time, model-based approaches such as Population-Based Incremental Learning (PBIL) provide an effective mechanism for capturing global structural information from high-quality solutions [2]. By maintaining a probability vector that reflects the distribution of promising solutions, PBIL offers a compact and interpretable representation of the search landscape [2]. However, in traditional PBIL frameworks, this probabilistic model is primarily used for sampling new solutions, rather than guiding the internal decision-making processes of evolutionary operators [2, 17].

This leads to a key observation: although both evolutionary search and probabilistic learning generate valuable information, they are often decoupled, resulting in inefficient use of available knowledge. In particular, there is a lack of mechanisms to utilise the learned probability distribution to guide fine-grained search decisions.

Motivated by this gap, this project explores how probabilistic knowledge can be directly integrated into evolutionary operators. Rather than treating PBIL as a standalone optimisation method, we leverage its probability vector as a source of guidance for operator design.

This motivation leads to the central idea of this work: establishing a co-evolutionary interaction between model learning and evolutionary search.

3 Related Work

3.1 Evolutionary and Memetic Algorithms

Evolutionary algorithms (EAs), particularly Genetic Algorithms (GAs), are among the most established metaheuristic approaches for combinatorial optimisation [13, 9]. Their effectiveness comes from maintaining a population of candidate solutions and iteratively improving that population through selection, crossover, and mutation. In binary optimisation, this framework is especially natural because candidate solutions can be represented directly as bitstrings, allowing variation operators to act at the level of individual decisions.

However, although standard EAs are effective at global exploration, their variation operators are usually stochastic and only weakly informed by search history. The search process may therefore spend substantial effort exploring regions that are structurally unpromising. This limitation is one reason why Memetic Algorithms (MAs) were introduced. MAs extend the evolutionary framework by hybridising population-based search with local search, thereby improving exploitation after variation [24, 16, 25]. Rather than relying on crossover and mutation alone, a memetic algorithm can refine offspring before replacement, often producing better-quality solutions and faster convergence on difficult combinatorial problems [16].

The literature on memetic search also makes clear that local search is not simply an add-on, but a core design choice that strongly influences performance. Different neighbourhood structures, move-acceptance criteria, and integration schedules can change the balance between intensification and diversification [16, 25]. More generally, hybrid metaheuristics have been studied as a broader class of methods that combine complementary search principles in order to improve robustness and solution quality [32]. This is directly relevant to the present project because the proposed framework remains memetic in structure: it retains a population-based evolutionary loop, but seeks to improve how local refinement and variation are guided.

3.2 Estimation of Distribution Algorithms and PBIL

A different line of research emerged through Estimation of Distribution Algorithms (EDAs), which replace or reinterpret traditional evolutionary variation through probabilistic modelling [17, 29]. Instead of generating new candidates only through crossover and mutation, EDAs learn a probability model from promising solutions and then sample new solutions from that model. This shifts the search process from purely operator-driven variation toward explicit learning of structural regularities in good solutions [17, 11].

Among the earliest and most influential methods in this family is Population-Based Incremental Learning (PBIL) [2]. PBIL represents the search state as a probability vector, where each component captures the estimated likelihood that a bit should take value 1 in promising solutions. This makes PBIL lightweight, memory-efficient, and naturally suited to binary optimisation. Related early work on EDA development includes distribution-based search over binary parameters [26], compact population models such as the compact genetic algorithm [10], and dependency-based probabilistic modelling that goes beyond purely independent bits [3].

Although EDAs and PBIL provide a powerful mechanism for capturing global structural information, their learned models are usually used for sampling new solutions rather than for directly guiding the fine-grained behaviour of search operators. In other words, the model influences what new solutions are generated, but not necessarily how local search should prioritise variables or how crossover should decide which bits deserve recombination. This is the main limitation that motivates the present dissertation. The key question is not whether PBIL can replace evolutionary search, but whether its probability vector can be embedded directly inside a memetic framework so that model learning and operator behaviour become tightly coupled.

3.3 Hybrid and Learning-Guided Search

A major trend in modern optimisation is the development of hybrid and learning-guided search methods. At a general level, hybrid metaheuristics combine two or more optimisation principles in order to exploit their complementary strengths [32]. In evolutionary computation, this often means combining population-based search with local refinement, adaptive operator control, or problem-specific learning mechanisms. In a wider methodological sense, machine learning has also been increasingly used in combinatorial optimisation to guide decisions that would otherwise be made by handcrafted heuristics [5].

Within evolutionary computation, a particularly relevant line of work studies model-building approaches that use learned structure to improve variation. For example, probabilistic model building has been used to combine crossover and mutation more intelligently, so that recombination is less disruptive and more consistent with underlying problem structure [20]. This is closely related in spirit to the present work. However, such studies generally do not investigate the use of a simple PBIL probability vector as a shared guidance signal for both local search and crossover within a unified Memetic Algorithm.

Therefore, while hybrid and learning-guided search are well established, the literature still leaves a gap at the operator level. Many approaches learn at the level of population generation, algorithm control, or solver selection, but relatively few embed a lightweight probabilistic model directly into bit-level local-search ordering and crossover eligibility inside one co-evolutionary framework. This is precisely the gap that this dissertation addresses.

3.4 Population Update, Elitism, and Learning Scope in Evolutionary and Probabilistic Search

Besides operator design, the way information is carried from one generation to the next is also fundamental in evolutionary optimisation. In standard evolutionary search, survivor selection, replacement, and elitism determine how strongly the algorithm preserves good solutions while maintaining enough diversity for continued exploration. Elitist designs are widely recognised as a powerful mechanism for protecting high-quality individuals and stabilising progress [7]. Although such ideas are often discussed in multi-objective settings, the broader principle is directly relevant to single-objective evolutionary and memetic search as well.

In memetic algorithms, update strategy matters at two levels. First, it affects which individuals survive after local refinement. Second, it changes how strongly the local-search-improved offspring influence the future direction of the population. Case studies in memetic optimisation show that performance depends not only on the presence of local search but also on how the refined solutions are integrated into the

population [16, 27]. Replacement is therefore part of the search bias rather than a purely administrative stage.

In PBIL and related EDAs, this issue becomes even more explicit because the update mechanism determines how the probabilistic model is learned from the current search state. PBIL updates a probability vector toward selected promising solutions [2], while related probabilistic approaches such as the compact genetic algorithm and other EDA families show that the strength of selection, the selected subset, and the model structure all materially affect convergence behaviour [29, 10, 11]. The difference between learning from one elite solution and learning from a broader subset is therefore not trivial: it changes whether the model acts as a focused intensification mechanism or a smoother population summary.

This point is reinforced by work on dynamic and non-stationary optimisation, where PBIL and UMDA variants have been extended using memory schemes and diversity-aware update rules [35, 34]. Although these studies are not solving the same problem as this dissertation, they demonstrate an important principle: the update rule of a probabilistic model is itself a major algorithm-design decision. For the present work, this provides a natural literature basis for studying PBIL learning scope, Top-K update behaviour, and the co-evolutionary relationship between the population and the probability vector.

3.5 Binary Representation, Binarisation, and Cross-Domain Binary Optimisation

The binary problem domain is broader than the two benchmark classes used in this dissertation. Many optimisation problems can be expressed in terms of yes-no, selected-not selected, active-inactive, or on-off decisions, making binary representations widely applicable across combinatorial optimisation and engineering design [28]. Some of these problems are inherently binary, such as MAX-SAT and 0–1 Knapsack. Others arise by transforming continuous search procedures into binary decision processes.

A classic example is Binary Particle Swarm Optimisation (BPSO), which adapts a continuous swarm method to binary spaces through a probabilistic bit-update mechanism [15]. Since then, a large body of research has investigated binary metaheuristics constructed by applying transfer functions or other mapping schemes to originally continuous algorithms [28]. Survey evidence suggests that transfer-function-based binarisation remains one of the dominant design patterns in this literature [28]. More recent work continues to refine these mechanisms through improved transfer functions [4], time-varying transfer schemes [18], and hybrid binary optimisers for feature selection and related tasks [1].

This literature is relevant because it shows that binary optimisation is not restricted to one or two domains. At the same time, it also highlights an important methodological issue: there is often a tension between variable-level meaning and bit-level operator action. When a continuous algorithm is binarised, or when a non-binary problem is encoded into bits, the resulting bit flips may not correspond cleanly to meaningful local changes in the original problem. This can weaken locality and make operator behaviour harder to interpret.

That issue is directly connected to the present dissertation. One of the motivations for studying MAX-SAT and 0–1 Knapsack together is that both are genuinely binary problem domains, allowing PBIL-guided operators to be analysed under a common representation. At the same time, they differ enough structurally to test whether the same probabilistic guidance mechanism remains useful across domains. Thus, the broader binary-optimisation literature helps justify the cross-domain framing of this work, while also showing why representation alignment matters.

3.6 MAX-SAT and 0–1 Knapsack as Benchmark Binary Domains

The Maximum Satisfiability problem (MAX-SAT) and the 0–1 Knapsack problem are both classical NP-hard binary combinatorial optimisation problems [8]. They are therefore natural benchmark domains for a dissertation concerned with binary evolutionary search.

For MAX-SAT, the literature includes both exact and heuristic paradigms. Solver-based methods such as MiniMaxSAT [12] and Open-WBO [23] illustrate the strength of SAT-based reasoning and exact or near-exact optimisation mechanisms on structured instances. At the same time, heuristic and metaheuristic work has explored evolutionary and memetic methods for MAX-SAT [6]. This makes MAX-SAT a suitable test domain for studying whether probabilistic guidance improves operator-level behaviour inside a memetic framework, especially when the aim is not to replace exact solvers but to understand the role of guided search.

The 0–1 Knapsack problem has an equally rich literature. Classical treatments established the algorithmic foundations of the problem [22], while later work explored exact algorithms for harder instance

classes [21] and questioned where genuinely difficult knapsack instances lie [30]. More recent analysis has further emphasised the importance of instance diversity and benchmark hardness when evaluating algorithms [31]. In parallel, genetic and memetic approaches have been studied for multidimensional and related knapsack variants [19, 33].

Taken together, MAX-SAT and 0–1 Knapsack form a useful pair for this dissertation. Both are binary problems, but they differ in how bit decisions interact. In MAX-SAT, each bit contributes through clause satisfaction, while in knapsack each bit contributes through profit-weight trade-offs under a global feasibility constraint. This contrast makes the pair particularly suitable for testing whether a common PBIL-guided memetic design offers domain-general value or whether its effect depends strongly on problem structure.

3.7 Positioning of This Work

This dissertation is positioned at the intersection of memetic search, probabilistic modelling, update-strategy design, and cross-domain binary optimisation. It builds on the EA and MA tradition of population-based variation with local refinement [16, 25, 24], while also drawing on EDA and PBIL ideas that explicitly model promising solution structure [2, 17, 29, 11].

However, the project differs from standard PBIL because the probability vector is not used only for sampling new individuals. It differs from broader hybrid-search literature because the model is embedded directly into operator behaviour rather than being used only for high-level control. It also differs from generic binary-metaheuristic work because it does not focus on binarising a continuous algorithm through transfer functions. Instead, it studies how a simple probabilistic model can guide local-search ordering, crossover eligibility, and learning scope inside a unified Memetic Algorithm operating on genuinely binary domains.

In this sense, the project is closest to a co-evolutionary interpretation of algorithm design, where the population and the probability vector learn from each other over time. The evolutionary process generates and refines candidate solutions; the PBIL model summarises structural tendencies in the better solutions; and that learned structure then feeds back into later operator decisions. This creates a tighter connection between global model learning and local operator behaviour than is usually seen in standard PBIL or in conventional memetic algorithms. It is this operator-level integration, together with the study of learning-scope update strategy and cross-domain binary generality, that defines the main contribution of the dissertation.

4 Description of Work

This project develops a unified co-evolutionary framework for binary combinatorial optimisation by integrating a Population-Based Incremental Learning (PBIL) model with a Memetic Algorithm (MA) [2, 25]. The framework combines population-based evolutionary search with probabilistic model-based learning [29], allowing information extracted from high-quality solutions to guide subsequent search decisions.

At a high level, the system contains two interacting components: an evolutionary search process that maintains and improves a population of candidate solutions, and a PBIL probability vector (PV) that captures bit-level statistical patterns from high-quality individuals. The evolutionary component generates new solutions through selection, variation, and local improvement, while the PBIL component continuously updates its probability model from the current search state.

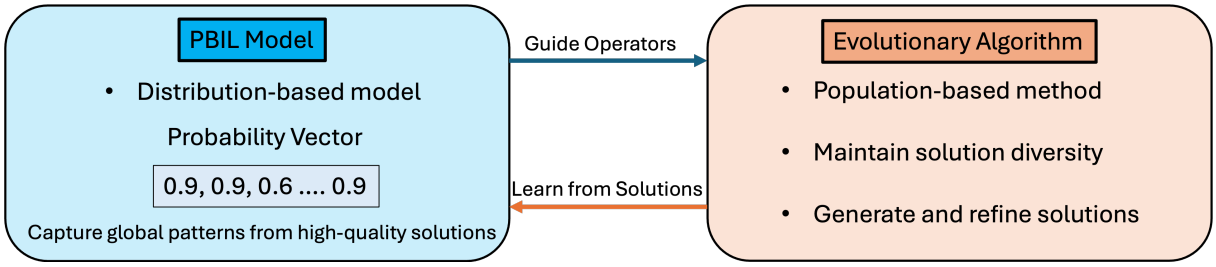


Figure 2: Conceptual interaction between the PBIL model and evolutionary operators through learning and guidance

The key idea of the framework is that these two components are not used independently. The PBIL model learns from the solutions produced by the evolutionary process, and the learned probability vector is then used to guide search operators, particularly local search and crossover. This creates a feedback loop in which probabilistic learning and evolutionary search reinforce each other over time.

Unlike conventional PBIL-based approaches, where the probabilistic model is mainly used to sample new solutions, the proposed framework emphasises operator-level guidance. The learned probability vector is used to influence how solutions are refined and recombined, transforming standard stochastic operators into more informed search mechanisms.

5 Methodology

5.1 Solution Representation and Objective Function

The proposed framework adopts a unified binary representation across all problem domains. Each candidate solution is encoded as a bitstring

$$\mathbf{x} = (x_1, x_2, \ldots, x_n), \quad x_i \in \{0, 1\} \tag{1}$$

This representation allows all evolutionary operators, including crossover, mutation, and local search, to operate consistently at the bit level regardless of the underlying optimisation problem. As a result, the same high-level search framework can be applied across different binary combinatorial domains while changing only the interpretation of each bit and the objective evaluation.

For the MAX-SAT problem, each bit corresponds to a Boolean decision variable. The optimisation objective is to maximise the number of satisfied clauses, which is equivalently handled in this framework as minimising the number of unsatisfied clauses.

For the 0–1 knapsack problem, each bit indicates whether an item is selected. Given n items with profits p_i and weights w_i, and a knapsack capacity C, the standard objective is:

$$\text{maximise} \quad \sum_{i=1}^{n} p_i x_i \quad \text{subject to} \quad \sum_{i=1}^{n} w_i x_i \leq C \tag{2}$$

To preserve a unified optimisation structure without introducing repair operators, constraint handling is implemented using a penalty-based formulation:

$$f(x) = -\sum_{i=1}^{n} p_i x_i + \lambda \cdot \max\left(0,\ \sum_{i=1}^{n} w_i x_i - C\right) \tag{3}$$

where C is the knapsack capacity and λ is a penalty coefficient. This converts the constrained maximisation problem into a minimisation objective while penalising infeasible solutions in proportion to the degree of constraint violation.

The penalty coefficient λ is derived from the average item density of each instance, defined as

$$\lambda_{\text{auto}} = \frac{1}{n} \sum_{i=1}^{n} \frac{p_i}{w_i} \tag{4}$$

where p_i/w_i is the profit-to-weight ratio (density) of item i. Using the average density as a baseline ensures that λ scales naturally with the value structure of each instance rather than relying on an arbitrary constant. The final penalty coefficient is then $\lambda = m \times \lambda_{\text{auto}}$, where m is an integer multiplier. The choice of $m = 2$ is justified by a penalty-multiplier comparison in Section 8.8.2.

Overall, this unified representation ensures that variation and improvement operators can be designed once at the bit level and then reused across both problem domains.

5.2 PBIL Probability Vector

The PBIL probability vector (PV) provides a compact probabilistic representation of promising solution structure within the proposed framework. For a binary solution of length n, the PV is defined as

$$\mathbf{p} = (p_1, p_2, \ldots, p_n) \tag{5}$$

where each $p_j \in [0, 1]$ denotes the learned probability of assigning value 1 at bit position j. Rather than storing a single candidate solution, the PV summarises population-level bitwise tendencies observed in high-quality solutions.

At initialization, all entries are set to 0.5, representing a neutral prior in which neither 0 nor 1 is preferred at any bit position. As optimisation progresses, the vector becomes increasingly informative by capturing recurring structural patterns from good solutions.

Within the proposed framework, the PV acts as a lightweight model that complements population-based search rather than replacing it. It provides a compact summary of learned bitwise information that can later be reused to guide search operators. This representation is especially suitable for the problem domains considered in this project, since both MAX-SAT and 0–1 knapsack use direct binary encodings.

5.3 PBIL Probability Vector Update Mechanism

The PV is updated once per generation within the PBIL-guided memetic algorithm. The update is performed after offspring generation, mutation, local search, and replacement, so that the model is learned from the current surviving population after selection pressure has already been applied.

To update the PV, the population is first ranked according to solution quality, and the best K individuals are selected as the learning source. The parameter K controls the scope of learning. When $K = 1$, the update is based only on the single best solution. When $K > 1$, the update is based on the average bit pattern of the top-K solutions.

For each bit position j, the proportion of selected individuals containing bit value 1 is computed as the update target:

$$\text{target}_j = \frac{1}{K} \sum_{i \in \text{TopK}} x_{i,j} \tag{6}$$

where $x_{i,j}$ denotes the value of bit j in solution i. The PV is then moved towards this target using learning rate α:

$$p_j \leftarrow (1 - \alpha) p_j + \alpha \cdot \text{target}_j \tag{7}$$

This update rule performs an exponential moving average between the current probability estimate and the newly observed elite pattern. A smaller learning rate leads to slower but more stable adaptation, whereas a larger learning rate makes the PV respond more rapidly to recent search outcomes.

To reduce premature convergence, each updated probability is clipped to a bounded interval. In the present implementation, PV values are restricted to $[0.05, 0.95]$. This prevents any bit from becoming fully deterministic too early, preserving a limited degree of uncertainty that remains useful for later variation and guidance.

Once updated, the PV is reused in the next generation to guide the search operators described in the following subsections.

5.4 PBIL-Guided Local Search

Local search is used within the proposed memetic framework to intensify search around newly generated offspring. Since both MAX-SAT and 0–1 knapsack are represented as binary strings, local improvement can be performed through single-bit modifications.

5.4.1 Baseline Bit Hill Climbing Framework

Two standard bit hill climbing strategies are considered as local-search baselines: Steepest Descent Hill Climbing (SDHC) and Davis's Bit Hill Climbing (DBHC). SDHC evaluates all possible one-bit flips and selects the best improving move at each iteration. In contrast, DBHC visits bit positions in a one-pass order and accepts a move immediately when it satisfies the chosen acceptance rule. In the baseline implementation, this visitation order is generated by a random permutation of the bit positions.

Compared with SDHC, DBHC has lower computational overhead and is more naturally suited to incorporating an externally defined bit-priority order. For this reason, DBHC is adopted as the backbone of the proposed PBIL-guided local-search variants. In these variants, the underlying single-bit-flip mechanism and acceptance rule remain unchanged; only the bit visitation order is replaced by a priority derived from the PBIL probability vector.

5.4.2 PBIL-Guided DBHC with Conflict-Based Priority

The first proposed variant prioritises bit positions according to their conflict with the current PBIL model. The intuition is that local search should first examine variables whose current values are least aligned with the probabilistic preference learned from high-quality solutions.

For a current solution x and probability vector p, the conflict priority at bit position j is defined as

$$\text{priority}_j = \begin{cases} 1 - p_j, & \text{if } x_j = 1 \\ p_j, & \text{if } x_j = 0 \end{cases} \tag{8}$$

This score is large when the current bit value disagrees with the model preference. For example, if $x_j = 1$ but p_j is small, then the current solution assigns 1 where the model tends to favour 0, indicating strong conflict. Likewise, if $x_j = 0$ but p_j is large, the bit is also in conflict with the learned preference. Bit positions are therefore sorted in descending order of this priority so that the most conflicting positions are examined first.

This produces a model-alignment form of local search: instead of exploring bits in a random order, the algorithm first inspects positions where the current solution most strongly disagrees with the structural tendencies learned by the PBIL model.

Algorithm 1 PBIL-Guided DBHC with Conflict-Based Priority

Require: Current solution $x \in \{0,1\}^n$, PBIL probability vector $p \in [0,1]^n$, objective function $f(\cdot)$ to be minimised
Ensure: Locally improved solution x

$F \leftarrow f(x)$
Create arrays `indices` $\leftarrow [1, 2, \ldots, n]$ and `priority`$[1..n]$
for $j \leftarrow 1$ **to** n **do**
    if $x_j = 1$ **then**
        `priority`$[j] \leftarrow 1 - p_j$
    else
        `priority`$[j] \leftarrow p_j$
    end if
end for
Sort `indices` in descending order of `priority`
for each $j \in$ `indices` **do**
    Flip bit x_j to obtain candidate x'
    $F' \leftarrow f(x')$
    if $F' \leq F$ **then**
        $x \leftarrow x'$
        $F \leftarrow F'$
    else
        Revert bit x_j in x
    end if
end for
return x

5.4.3 PBIL-Guided DBHC with Entropy-Based Priority

The second proposed variant prioritises bit positions according to model uncertainty. The main idea is that probabilities close to 0.5 indicate that the PBIL model remains uncertain about the preferred value at that position, whereas probabilities closer to 0 or 1 indicate stronger confidence. Rather than enforcing the current model preference, this strategy focuses local search on unresolved positions where productive changes may still be possible.

Uncertainty at bit position j is measured as

$$\text{uncertainty}_j = |p_j - 0.5| \tag{9}$$

Smaller values correspond to higher uncertainty. Bit positions are therefore sorted in ascending order of this quantity so that the most uncertain bits are examined first.

This strategy treats uncertainty as an indicator of incomplete structure in the learned model. By directing local search toward positions where the model has not yet formed a strong preference, the method allows search effort to concentrate on variables that may still admit meaningful improvement.

Algorithm 2 PBIL-Guided DBHC with Entropy-Based Priority

Require: Current solution $x \in \{0,1\}^n$, PBIL probability vector $p \in [0,1]^n$, objective function $f(\cdot)$ to be minimised
Ensure: Locally improved solution x
$F \leftarrow f(x)$
Create arrays `indices` $\leftarrow [1, 2, \ldots, n]$ and `uncertainty`$[1..n]$
for $j \leftarrow 1$ **to** n **do**
    `uncertainty`$[j] \leftarrow |p_j - 0.5|$
end for
Sort `indices` in ascending order of `uncertainty`
for each $j \in$ `indices` **do**
    Flip bit x_j to obtain candidate x'
    $F' \leftarrow f(x')$
    if $F' \leq F$ **then**
        $x \leftarrow x'$
        $F \leftarrow F'$
    else
        Revert bit x_j in x
    end if
end for
return x

5.4.4 Summary of the Local-Search Design

Both proposed variants preserve the basic DBHC framework and differ only in how the bit visitation order is constructed. The conflict-based variant promotes alignment between the current solution and the learned PBIL model, while the entropy-based variant emphasises variables for which the model remains uncertain. In this way, the PBIL probability vector is used not to replace local search, but to inform where local improvement should be applied first.

5.5 PBIL-Guided Crossover

Crossover is used within the proposed evolutionary framework to recombine information from two parent solutions and generate new offspring. Since both MAX-SAT and 0–1 knapsack are represented as binary strings, bitwise recombination provides a natural mechanism for mixing parental information.

5.5.1 Baseline Uniform Crossover

Uniform crossover is adopted as the baseline recombination operator. Given two parent solutions, the operator first copies them into two offspring. It then scans all bit positions and, whenever the parents differ at a position, swaps the corresponding offspring bits with probability 0.5. Positions where the two parents already share the same value remain unchanged.

Compared with more structured recombination mechanisms, uniform crossover is simple and unbiased. However, it does not distinguish between uncertain and well-established positions in the current search process, and may therefore introduce unnecessary disruption.

5.5.2 PBIL-Guided Entropy-Restricted Uniform Crossover

To reduce this disruption, a PBIL-guided entropy-restricted uniform crossover is introduced. The key idea is to permit recombination only at bit positions where the PBIL model remains uncertain. For each bit position j, model confidence is defined as

$$\mathrm{conf}_j = 2|p_j - 0.5|. \tag{10}$$

A small value of conf_j indicates that p_j is close to 0.5 and that the model has not yet developed a strong preference at that position. Given a threshold τ, crossover is therefore allowed only when $\text{conf}_j < \tau$.

If the two parents differ at such a position, the corresponding offspring bits are swapped with probability 0.5, as in standard uniform crossover. Otherwise, the position is preserved unchanged. In this way, recombination is concentrated on uncertain regions of the solution representation, while more stable learned structure is protected from unnecessary disturbance.

Algorithm 3 PBIL-Guided Entropy-Restricted Uniform Crossover

Require: Parent solutions $P^{(1)}, P^{(2)} \in \{0,1\}^n$, PBIL probability vector $p \in [0,1]^n$, threshold τ
Ensure: Offspring solutions $C^{(1)}, C^{(2)}$

```
C^(1) ← P^(1)
C^(2) ← P^(2)
for j ← 1 to n do
    if P_j^(1) = P_j^(2) then
        continue
    end if
    conf_j ← 2|p_j − 0.5|
    if conf_j < τ then
        With probability 0.5, swap C_j^(1) and C_j^(2)
    end if
end for
return C^(1), C^(2)
```

6 Design

6.1 Overall Framework Design

The proposed system adopts a unified co-evolutionary architecture that integrates a Population-Based Incremental Learning (PBIL) model with a Memetic Algorithm (MA). The framework follows a fixed evolutionary pipeline comprising parent selection, variation, local search, replacement, and probabilistic model update, as illustrated previously in Figure 1.

At each generation, candidate solutions are first evolved through the standard evolutionary cycle. After replacement, the PBIL model is updated using high-quality solutions from the current population, allowing the probability vector to reflect the latest search state. The updated model then provides guidance to later search operators, forming a consistent feedback loop between population-based search and probabilistic learning.

This design separates **solution evolution** from **model learning** while allowing both components to interact within the same iterative process. As a result, the framework supports controlled operator integration while preserving a unified optimisation structure across problem domains.

6.2 Modular Component Design

A central design feature of the proposed framework is the decomposition of the evolutionary process into independent and interchangeable components. The main modules include parent selection, crossover, mutation, local search, replacement, and the PBIL model update mechanism.

Each module is implemented separately so that individual operators can be configured or replaced without altering the overall optimisation pipeline. This design supports controlled experimentation, since the effect of a specific operator can be evaluated while the remaining components are kept fixed. As a result, the framework provides both implementation flexibility and a consistent basis for fair comparison across algorithm variants.

6.3 Cross-Domain Unified Design

The proposed framework is designed to operate consistently across multiple binary optimisation domains, including MAX-SAT and the 0–1 Knapsack problem. In both cases, the same high-level optimisation structure is retained, comprising population-based search, variation, local search, replacement, and

PBIL model update. This preserves a unified co-evolutionary pipeline while allowing the framework to be applied to different problem settings.

The main domain-level differences arise in the evaluation of candidate solutions and the interpretation of binary decision variables. In MAX-SAT, solution quality is determined by clause satisfaction, whereas in the 0–1 Knapsack problem, solutions are evaluated using a profit-based objective with penalty handling for capacity violations. Despite these differences, both domains remain compatible with the same binary representation and operator-level search structure. This design supports consistency across domains and helps distinguish the contribution of the search framework from problem-specific evaluation details.

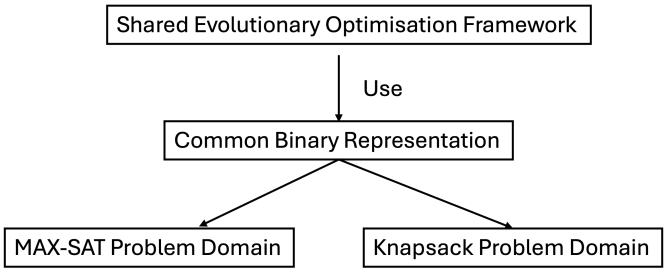


Figure 3: Shared optimisation structure and binary representation across the MAX-SAT and 0–1 Knapsack domains

6.4 Design Rationale

The framework design is guided by four main principles: modularity, consistency, fairness, and extensibility. Modularity is achieved by separating the optimisation process into interchangeable components, which allows individual operators to be configured or replaced without changing the overall pipeline. This supports controlled experimentation and makes it easier to study the contribution of specific search mechanisms.

Consistency is maintained by preserving the same high-level optimisation structure across problem domains. This allows MAX-SAT and 0–1 Knapsack to be studied within a common framework, improving comparability between results. Fairness is supported by fixing all components except the one under investigation, so that performance differences can be attributed more clearly to the operator or mechanism being evaluated. Extensibility is provided through the modular structure, which allows new operators or probabilistic model variants to be integrated with minimal changes to the rest of the framework.

Taken together, these design principles provide a robust and flexible basis for studying the interaction between probabilistic learning and evolutionary search in binary optimisation.

7 Implementation

7.1 System Implementation Overview

The proposed framework was implemented as a modular PBIL-guided memetic algorithm in Java. The system follows a unified optimisation pipeline consisting of parent selection, variation, local search, replacement, and probabilistic model update. Each stage is implemented as a separate component, allowing individual operators to be configured or replaced without altering the overall execution flow.

This implementation structure supports controlled comparison between algorithm variants. In particular, PBIL-guided operators can be introduced within the same framework while the remaining components are kept unchanged, ensuring that performance differences can be attributed more clearly to the mechanism under investigation.

7.2 Baseline Configuration and Component Instantiation

A common baseline configuration was adopted across experiments to provide a consistent reference point for comparison. Tournament selection was used for parent selection, uniform crossover was used as the baseline recombination operator, and bit-wise mutation was applied to maintain diversity during search. For local search, Davis's Bit Hill Climbing (DBHC) was used as the primary baseline, with PBIL-guided variants introduced as alternative operator instantiations. Population replacement was

implemented using trans-generational replacement with elitism in order to preserve high-quality solutions across generations.

This fixed configuration ensured that comparisons between search variants were carried out under consistent conditions. As a result, improvements observed in the experiments can be interpreted more reliably as arising from the introduced PBIL-guided mechanisms rather than unrelated implementation changes.

7.3 Parameter Settings

To ensure consistency and reproducibility, the same core parameter settings were used throughout the study:

- population size: 50
- number of generations: 60

To ensure the statistical reliability of the results, each configuration was evaluated across 31 independent runs. Apart from parameters examined in dedicated experiments, the remaining settings were kept fixed across all algorithm configurations. This helped maintain fair comparison between variants while reducing unnecessary experimental variation.

8 Evaluation

8.1 Benchmark Datasets

8.1.1 MAX-SAT Benchmark Set

MAX-SAT was used as the primary benchmark domain for evaluating the proposed PBIL-guided framework. To assess algorithm behaviour across a diverse range of problem settings, 12 benchmark instances were selected from multiple sources. These instances vary in both scale and structure, with the number of variables ranging from 250 to 744 and the number of clauses ranging from 1000 to 3500. This diversity provides a broader basis for evaluation than relying on a single instance and allows the comparative performance of different algorithm variants to be assessed under heterogeneous MAX-SAT conditions.

ID	Instance name	Source	Variables	Clauses
0	contest02-Mat26.sat05-457.reshuffled-07	[14]	744	2464
1	hidden-k3-s0-r5-n700-01-S2069048075.sat05-488.reshuffled-07	[14]	700	3500
2	hidden-k3-s0-r5-n700-02-S350203913.sat05-486.reshuffled-07	[14]	700	3500
3	parity-games/instance-n3-i3-pp	[14]	525	2276
4	parity-games/instance-n3-i3-pp-ci-ce	[14]	525	2336
5	parity-games/instance-n3-i4-pp-ci-ce	[14]	696	3122
6	highgirth/3SAT/HG-3SAT-V250-C1000-1	[14]	250	1000
7	highgirth/3SAT/HG-3SAT-V250-C1000-2	[14]	250	1000
8	highgirth/3SAT/HG-3SAT-V300-C1200-2	[14]	300	1200
9	MAXCUT/SPINGLASS/t7pm3-9999	[14]	343	2058
10	jarvisalo/eq.atree.braun.8.unsat	[14]	684	2300
11	highgirth/3SAT/HG-3SAT-V300-C1200-4	[14]	300	1200

Table 1: MAX-SAT Benchmark Instances

8.1.2 0–1 Knapsack Benchmark Set

To evaluate cross-domain generality, the proposed framework was further tested on 12 benchmark instances from the Pisinger 0–1 Knapsack dataset [30]. The selected instances cover four canonical correlation classes: uncorrelated, weakly correlated, strongly correlated, and inverse strongly correlated.

For each class, instances were chosen from different size bands in order to capture variation in both structural properties and problem scale. This selection allows the evaluation to examine whether the PBIL-guided mechanisms remain effective beyond MAX-SAT under different profit-weight relationships and instance sizes.

ID	File name	Class	Class description	Items (n)	r	Seed	Size band
1	pisinger_small__knapPI_1_200_10000_4.txt	1	Uncorrelated	200	10000	4	small
2	pisinger_small__knapPI_1_1000_10000_6.txt	1	Uncorrelated	1000	10000	6	medium
3	pisinger_small__knapPI_1_5000_10000_23.txt	1	Uncorrelated	5000	10000	23	large
4	pisinger_small__knapPI_2_200_10000_91.txt	2	Weakly Correlated	200	10000	91	small
5	pisinger_small__knapPI_2_1000_10000_25.txt	2	Weakly Correlated	1000	10000	25	medium
6	pisinger_small__knapPI_2_10000_10000_29.txt	2	Weakly Correlated	10000	10000	29	large
7	pisinger_small__knapPI_3_200_10000_50.txt	3	Strongly Correlated	200	10000	50	small
8	pisinger_small__knapPI_3_1000_10000_36.txt	3	Strongly Correlated	1000	10000	36	medium
9	pisinger_small__knapPI_3_10000_10000_12.txt	3	Strongly Correlated	10000	10000	12	large
10	pisinger_small__knapPI_4_50_10000_10.txt	4	Inverse Strongly Correlated	50	10000	10	small
11	pisinger_small__knapPI_4_500_10000_83.txt	4	Inverse Strongly Correlated	500	10000	83	medium
12	pisinger_small__knapPI_4_10000_10000_31.txt	4	Inverse Strongly Correlated	10000	10000	31	large

Table 2: 0–1 Knapsack Benchmark Instances

8.2 PBIL Probability-Vector Learning Behaviour

To illustrate how the PBIL model evolves during search, probability-vector (PV) convergence was analysed using both bitwise heatmaps and PV-entropy traces. This analysis was performed under the PBIL-active baseline configuration, using uniform crossover and baseline DBHC local search, in order to visualise how the probability model changes over generations under different learning-rate settings.

For a PBIL probability vector, each bit i has an associated probability p_i of taking value 1. The uncertainty of each bit can be measured using Shannon entropy:

$$h(p_i) = -p_i \ln(p_i) - (1 - p_i) \ln(1 - p_i) \tag{11}$$

and the total PV entropy is obtained by summing this quantity over all bits. In this study, natural logarithms are used, so entropy is measured in nats. A high PV entropy indicates that the model remains uncertain about many bit positions, whereas a low PV entropy indicates that the model has become more committed and converged toward particular values.

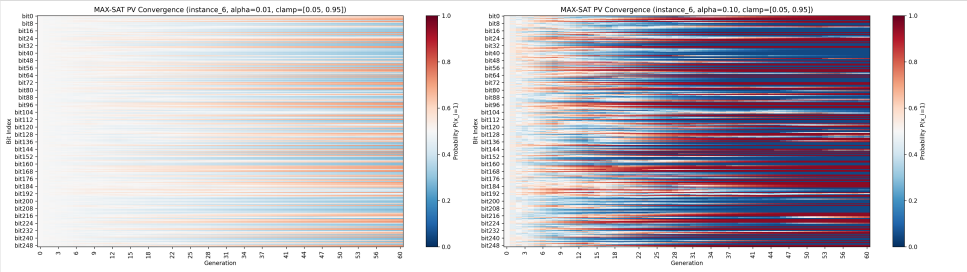


Figure 4: MAX-SAT probability-vector heatmaps for learning rates $\alpha = 0.01$ and $\alpha = 0.10$.

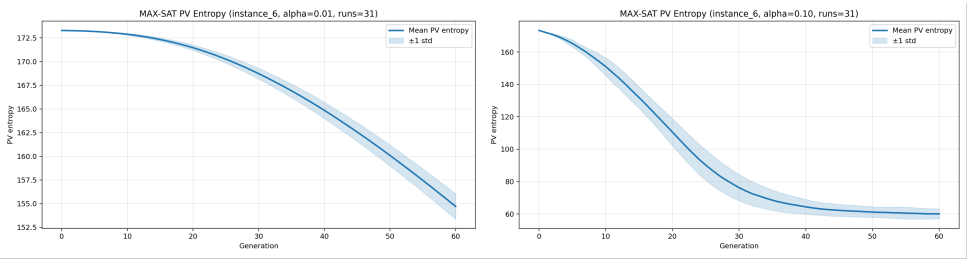


Figure 5: MAX-SAT probability-vector entropy traces for learning rates $\alpha = 0.01$ and $\alpha = 0.10$.

Figures 4 and 5 show the MAX-SAT PV heatmaps and entropy curves for learning rates $\alpha = 0.01$ and $\alpha = 0.10$, respectively. With $\alpha = 0.01$, the heatmap shows gradual and relatively smooth probability movement, with many bits remaining in intermediate regions for a longer part of the run. This is consistent with the entropy curve, which decreases slowly over generations, indicating stable and progressive reduction of uncertainty. In contrast, with $\alpha = 0.10$, the heatmap exhibits much faster probability polarisation, with many bits moving rapidly toward the probability bounds. The corresponding entropy curve drops sharply in the early generations, showing that the PV commits much more aggressively under the larger learning rate.

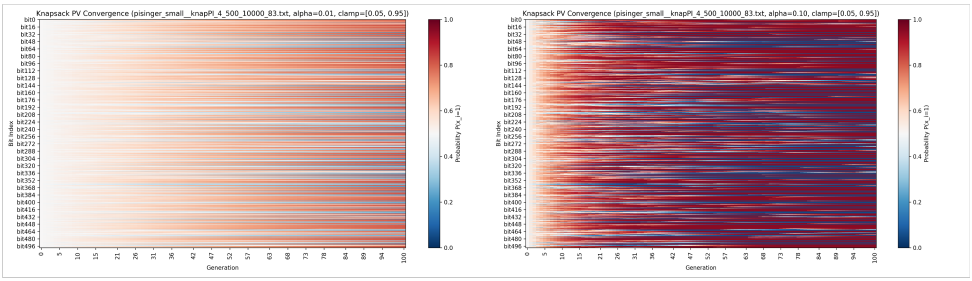


Figure 6: 0–1 knapsack probability-vector heatmaps for learning rates $\alpha = 0.01$ and $\alpha = 0.10$.

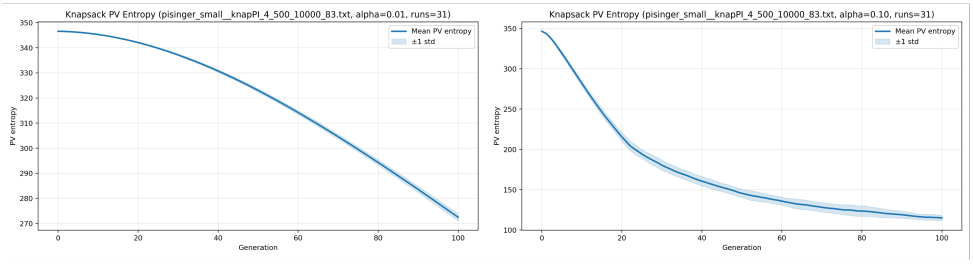


Figure 7: 0–1 knapsack probability-vector entropy traces for learning rates $\alpha = 0.01$ and $\alpha = 0.10$.

The same pattern is observed in the 0–1 knapsack domain, as shown in Figures 6 and 7. Under $\alpha = 0.01$, the PV again evolves gradually and retains more uncertainty throughout the run, while $\alpha = 0.10$ produces substantially faster entropy reduction and stronger early polarisation across many bit positions. This suggests that the qualitative learning behaviour of the PBIL model is consistent across both problem domains: smaller learning rates favour more stable adaptation, whereas larger learning rates encourage faster but more aggressive convergence.

Overall, the PV heatmaps and entropy traces provide a direct visual interpretation of PBIL learning dynamics. They show that the learning rate has a strong influence on how quickly the probability model loses uncertainty and commits to specific bit values, thereby motivating the later learning-rate sensitivity analysis.

8.3 Local Search Performance on MAX-SAT

This experiment evaluates whether PBIL guidance improves local search performance within the proposed memetic framework on MAX-SAT. Four local search variants were compared: **MA_DBHC_IE**, **MA_SDHC_IE**, **MA_PBIL_Guided_DBHC_IE**, and **MA_PBIL_Guided_DBHC_Entropy_IE**. The evaluation was conducted in two stages. First, a representative MAX-SAT instance was analysed in detail using 31 independent runs in order to examine both final-solution quality and convergence behaviour. Second, the comparison was extended to all 12 benchmark instances using an instance-wise ranking procedure based on median final fitness. Since MAX-SAT is treated as a minimisation problem in this framework, lower fitness values indicate better performance.

8.3.1 Performance on a Representative MAX-SAT Instance

Figure 8 shows the distribution of final best fitness values on Instance 6. The two strongest methods are **MA_DBHC_IE** and **MA_PBIL_Guided_DBHC_Entropy_IE**, both of which achieve relatively low final fitness with compact distributions across runs. The entropy-guided PBIL variant attains a slightly lower median, suggesting a small advantage on this instance. By contrast, **MA_PBIL_Guided_DBHC_IE**

performs worse than the baseline DBHC variant, while **MA_SDHC_IE** shows the weakest overall performance, with substantially higher final fitness values.This pattern is reinforced by the convergence curves in Figure 9.

Both **MA_DBHC_IE** and **MA_PBIL_Guided_DBHC_Entropy_IE** improve rapidly in the early generations and converge to the best final plateaus among the compared methods.

In contrast, **MA_PBIL_Guided_DBHC_IE** does not surpass the DBHC baseline, indicating that PBIL guidance alone is not sufficient to improve performance. **MA_SDHC_IE** converges much more slowly and remains at a clearly worse fitness level throughout the run.

Taken together, the representative-instance results suggest that the main benefit does not come from PBIL guidance in isolation, but from combining PBIL information with entropy-based variable ordering. This combination appears to improve both effectiveness and consistency.

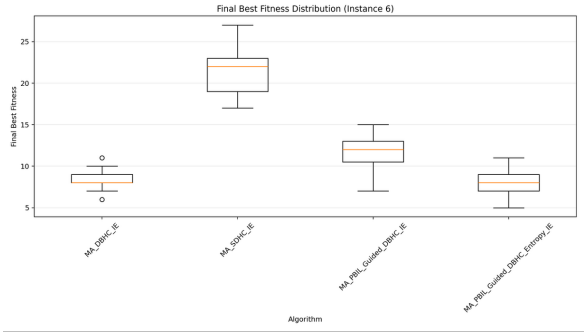


Figure 8: Distribution of final best fitness over 31 runs on MAX-SAT Instance 6.

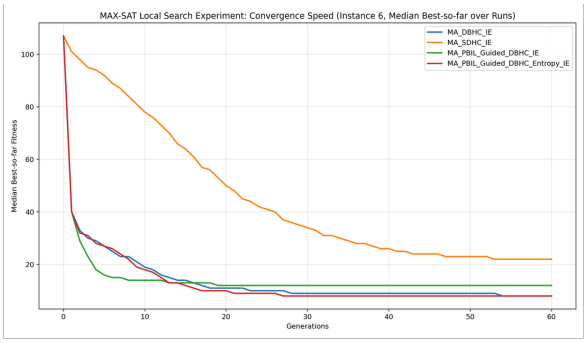


Figure 9: Median best-so-far fitness over generations on MAX-SAT Instance 6.

8.3.2 Performance Across 12 MAX-SAT Instances

To assess whether the instance-level observations generalise across the benchmark set, the same four local search variants were evaluated on all 12 MAX-SAT instances. Rather than ranking algorithms only after collapsing each instance to a single summary statistic, a paired-run ranking procedure was used. For each (instance, run) pair, the four algorithms were ranked according to final best fitness, with rank 1 assigned to the best-performing method and tied methods receiving the average tied rank. These run-level ranks were then averaged across runs for each instance, and the final overall score was computed as the average of the instance-level paired ranks.

The per-instance results are shown in Figure 10. Across most instances, **MA_PBIL_Guided_DBHC_Entropy_IE** achieves the lowest average paired rank, indicating that it outperforms the competing local search variants more consistently over repeated runs. **MA_DBHC_IE** is generally the second strongest method and remains competitive on several instances, although it is less consistent than the entropy-guided variant. **MA_PBIL_Guided_DBHC_IE** typically ranks third, while **MA_SDHC_IE** is ranked last on every instance.

Table 3 summarises the overall average paired rank across all 12 instances. **MA_PBIL_Guided_DBHC_Entropy_IE** achieves the best overall score (1.48), followed by **MA_DBHC_IE** (1.78), **MA_PBIL_Guided_DBHC_IE** (2.74), and **MA_SDHC_IE** (4.00). These results strengthen the same conclusion suggested by the representative-instance analysis: entropy-guided PBIL enhancement provides the most reliable improvement among the tested local search strategies.

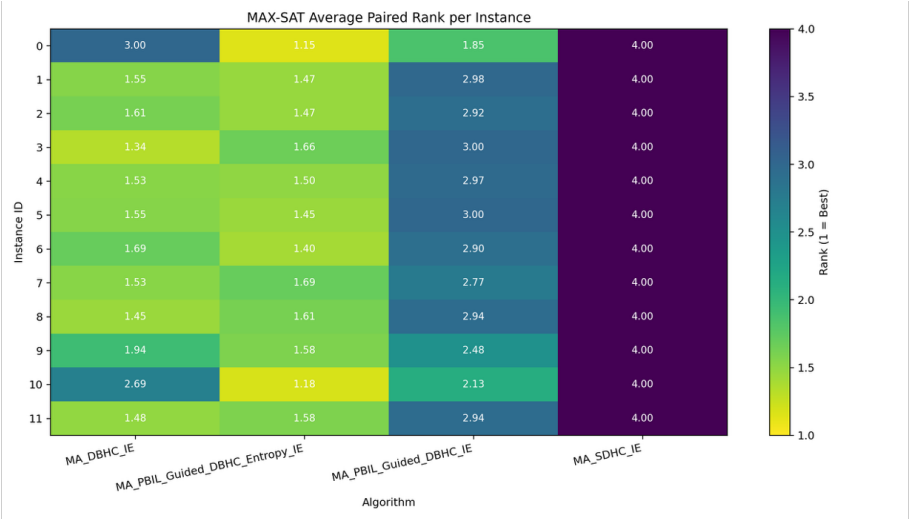


Figure 10: Per-instance average paired rank of the four local search variants across the 12 MAX-SAT benchmark instances. Ranks are computed within each (instance, run) pair and then averaged across runs. Lower ranks indicate better performance.

Algorithm	**Overall average paired rank**
MA_PBIL_Guided_DBHC_Entropy_IE	1.48
MA_DBHC_IE	1.78
MA_PBIL_Guided_DBHC_IE	2.74
MA_SDHC_IE	4.00

Table 3: Overall average paired rank of each local search variant across the 12 MAX-SAT benchmark instances. Lower values indicate better overall performance.

8.3.3 Discussion

The local search experiments reveal an important distinction. Incorporating PBIL information into DBHC does not, by itself, guarantee improved performance over the baseline. In the present results, **MA_PBIL_Guided_DBHC_IE** remains clearly weaker than **MA_DBHC_IE**, indicating that direct PBIL-guided prioritisation alone is insufficient.

By contrast, when PBIL guidance is combined with entropy-based bit ordering, the resulting method becomes the strongest local search variant across the MAX-SAT benchmark set. This suggests that the main value of PBIL in this context is not to enforce current model preference directly, but to identify uncertain bit positions where search effort is likely to be more useful.

The results also show that **MA_DBHC_IE** is a strong baseline. Its second-best overall rank indicates that simple first-improvement local search already matches the structure of MAX-SAT well. In contrast, **MA_SDHC_IE** performs consistently poorly, with an overall average paired rank of 4.00, suggesting that its greedy best-improvement behaviour is not well suited to this setting.

Overall, these results identify **MA_PBIL_Guided_DBHC_Entropy_IE** as the strongest and most reliable local search variant among those tested. Its advantage over the DBHC baseline is not only visible on the representative instance, but is also maintained more consistently across repeated runs on the full 12-instance benchmark set. For this reason, it is adopted as the preferred local search design in the subsequent experiments.

8.4 PBIL Learning-Rate Sensitivity Analysis

Before the main comparative experiments, a preliminary sensitivity study was conducted to identify a suitable PBIL configuration for the entropy-guided local search variant. The aim of this experiment was not to compare algorithms, but to determine a stable and defensible parameter setting for later evaluation.

The study was performed on MAX-SAT instance 6 using a controlled grid search over the PBIL learning rate (α) and the probability clamp lower bound (ϵ). The tested learning-rate values were

$$\alpha \in \{0.002, 0.005, 0.01, 0.02, 0.03, 0.05, 0.10, 0.20\}$$

and the tested clamp settings were

$$\epsilon \in \{0.01, 0.03, 0.05, 0.08, 0.12\}.$$

For each (α, ϵ) pair, 11 independent runs were executed while keeping all other algorithmic components fixed. Performance was measured using the mean final best objective value, where lower values indicate better performance.

Figure 11 shows the resulting tuning heatmap. Two main observations can be made. First, the learning rate (α) has a much stronger effect on performance than the clamp setting (ϵ). Second, the most reliable operating region lies in the low-to-moderate learning-rate range, with $\alpha = 0.01$ giving the best overall results across all tested ϵ values. Once $\alpha = 0.01$ is used, performance becomes relatively insensitive to the precise choice of ϵ.

Based on this result, $\alpha = 0.01$ was selected for the subsequent experiments. A moderate clamp setting was retained for the remaining evaluation, since the heatmap indicates that ϵ has only a limited effect when the learning rate is appropriately chosen.

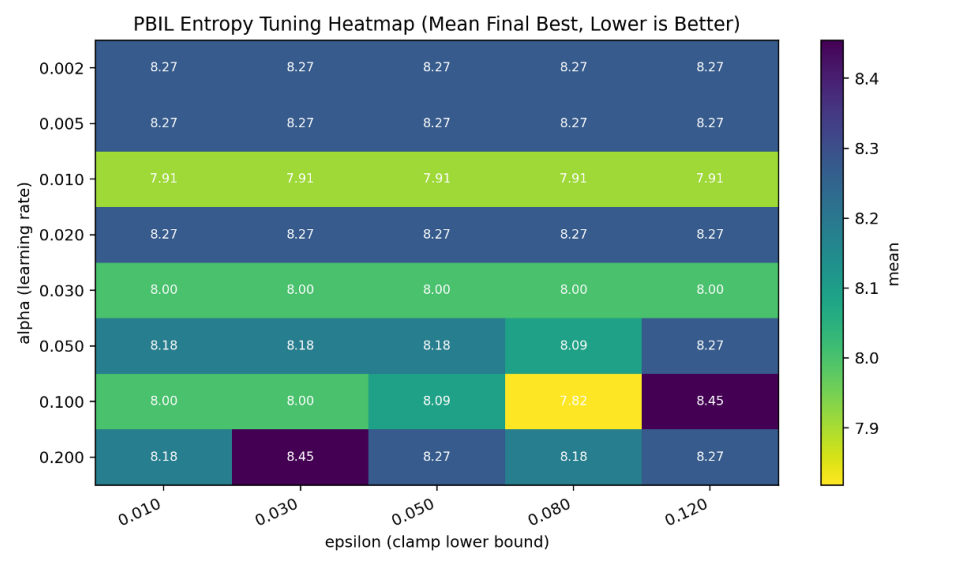


Figure 11: Mean final best objective value on MAX-SAT instance 6 across PBIL learning-rate and clamp settings for the entropy-guided local search variant. Lower values indicate better performance.

8.5 Crossover Performance on MAX-SAT

This experiment evaluates whether the proposed entropy-restricted uniform crossover (ER-UXO) improves performance within the PBIL-guided memetic framework on MAX-SAT. Unlike standard uniform crossover, ER-UXO uses the PBIL probability vector to identify uncertain bit positions and restricts recombination to those locations only. The aim is therefore not to increase mixing broadly, but to make crossover more selective and more consistent with the state of the learned probabilistic model.

The evaluation was conducted in four stages. First, the uncertainty threshold parameter (τ) was tuned on a representative MAX-SAT instance. Second, the crossover behaviour was analysed in detail on a single instance using convergence curves and final-best distributions. Third, the number of crossover-eligible bits was tracked over generations to examine how PBIL uncertainty changes the effective scope of recombination during search. Finally, the comparison was extended to all 12 MAX-SAT benchmark instances using an instance-wise ranking analysis.

8.5.1 ER-UXO Design and τ Sensitivity

Figure 12 shows the coarse-search effect of the ER-UXO uncertainty threshold (τ) on MAX-SAT performance. The best results occur at the smallest threshold values, which means that crossover is most

helpful when it is restricted to bits whose PBIL probabilities remain very close to 0.5. Even at this broader tuning stage, the overall pattern still points to a highly conservative crossover regime rather than to frequent or wide-ranging recombination.

As τ increases, performance generally deteriorates and moves closer to the standard uniform-crossover baseline. This suggests that ER-UXO is most useful as a focused exploratory mechanism on genuinely uncertain positions rather than as a broad mixing operator. In other words, the MAX-SAT results do not support aggressive crossover; instead, they show that recombination is beneficial only when applied selectively and conservatively.

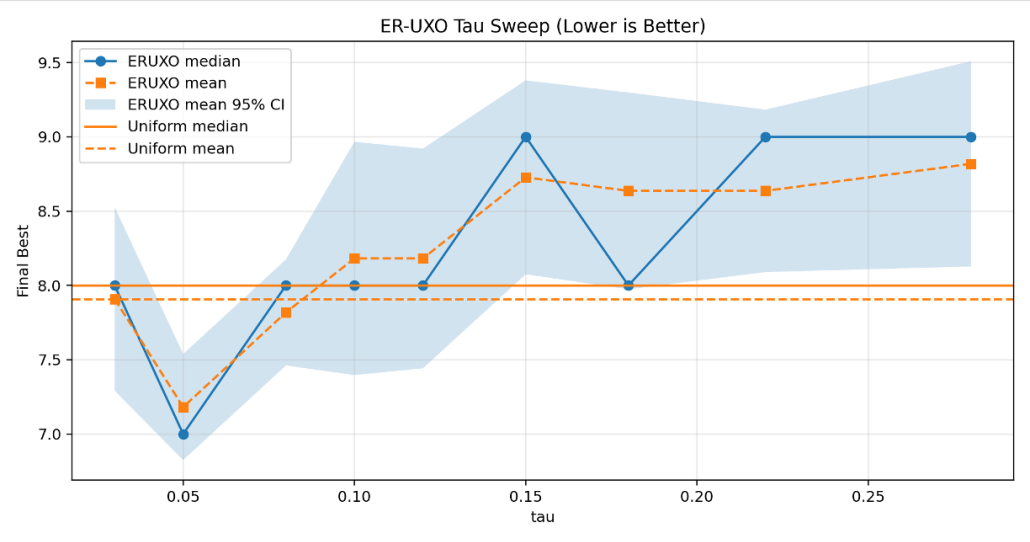


Figure 12: Coarse-search result for the ER-UXO uncertainty threshold (τ) on MAX-SAT Instance 6, compared with the standard uniform-crossover baseline. Lower final best objective values indicate better performance.

8.5.2 Representative-Instance Analysis

After fixing $\tau = 0.05$, crossover behaviour was analysed in detail on Instance 0. Four configurations were compared: standard uniform crossover with DBHC local search (**UNIFORM_DBHC**), ER-UXO with DBHC (**ERUXO_DBHC**), standard uniform crossover with entropy-guided local search (**UNIFORM_ENTROPY**), and ER-UXO with entropy-guided local search (**ERUXO_ENTROPY**).

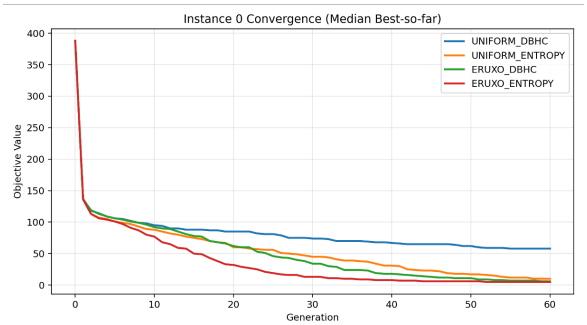


Figure 13: Median best-so-far objective value over generations on MAX-SAT Instance 0 for the compared crossover configurations.

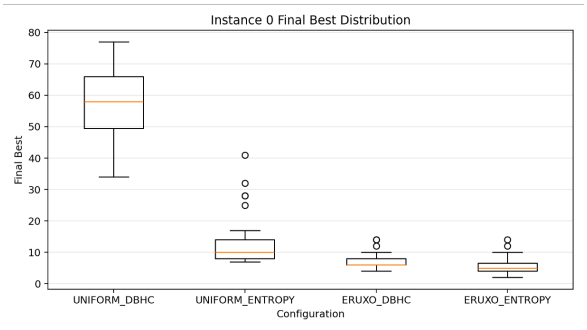


Figure 14: Distribution of final best objective values over 31 runs on MAX-SAT Instance 0 for the compared crossover configurations.

The convergence curves in Figure 13 show that **ERUXO_ENTROPY** improves most rapidly and reaches the best final plateau. **UNIFORM_ENTROPY** also performs strongly, while both DBHC-based crossover configurations remain weaker, especially **UNIFORM_DBHC**, which converges to a substantially worse final objective value. The same pattern is confirmed by the final-best boxplots in Figure 14, where **ERUXO_ENTROPY** achieves the lowest and most compact distribution overall.

These results show that the main contribution of ER-UXO is not to increase crossover activity, but to control it. The entropy-guided local search remains the main performance driver, while ER-UXO provides a further benefit by making recombination more selective and less disruptive.

8.5.3 Eligible-Bit Dynamics and PBIL Gating Effect

Figure 15 provides direct evidence for this mechanism by showing the number of crossover-eligible bits over generations for $\tau = 0.05$. At the start of search, almost all bit positions are eligible because the PV is initialised near maximum uncertainty. As learning progresses, the number of eligible bits decreases sharply, dropping from the full bit set to only a small residual subset by later generations.

This behaviour shows that the PBIL model acts as a gating mechanism for crossover. Early in the run, recombination remains available across many positions, allowing some exploratory mixing. Later, as the PV becomes more confident and more bit probabilities move away from 0.5, the active scope of crossover is progressively closed. As a result, crossover becomes naturally suppressed once the search begins to stabilise, helping the algorithm avoid unnecessary disruption of good partial assignments.

This mechanism supports the earlier τ-sensitivity result. The advantage of ER-UXO comes not from applying more crossover, but from restricting recombination to the limited set of genuinely uncertain bits where exploration is still justified.

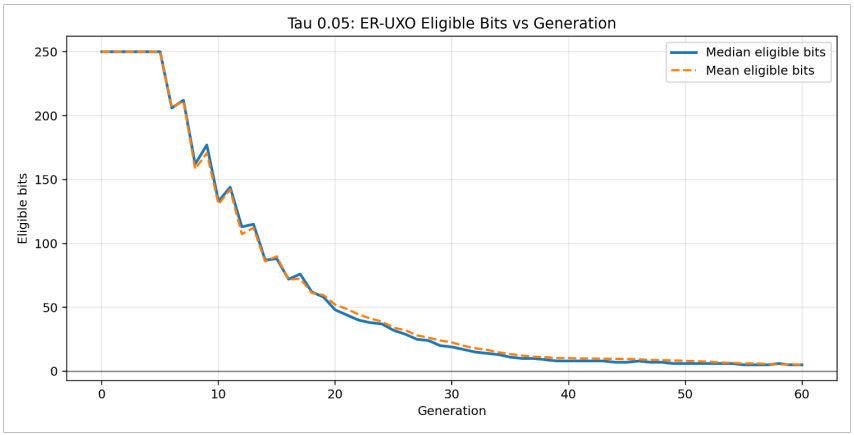


Figure 15: Mean and median number of crossover-eligible bits over generations for ER-UXO with $\tau = 0.05$ on MAX-SAT.

8.5.4 Performance Across 12 MAX-SAT Instances

To assess whether these observations generalise beyond a single instance, the same four crossover configurations were compared across all 12 MAX-SAT benchmark instances. For each instance, algorithms were ranked according to final performance, and the results were summarised using average ranks.

Figure 16 shows the per-instance average-rank heatmap, while Table 4 reports the overall average rank across all 12 MAX-SAT instances. Across the benchmark set, **ERUXO_ENTROPY** achieves the strongest overall performance, obtaining the best or joint-best ranking on most instances. **ERUXO_DBHC** remains competitive on several cases and ranks second overall, while **UNIFORM_ENTROPY** is weaker and **UNIFORM_DBHC** is the worst overall configuration.

Taken together, these results support the same conclusion as the representative-instance analysis: crossover is not uniformly beneficial in MAX-SAT, and its value depends strongly on how selectively it is applied. The strongest results are obtained when recombination is tightly restricted by PBIL uncertainty and combined with the entropy-guided local search variant. Table 4 confirms this pattern numerically, with **ERUXO_ENTROPY** achieving the best overall average rank of 1.88, followed by **ERUXO_DBHC** at 2.32, **UNIFORM_ENTROPY** at 2.67, and **UNIFORM_DBHC** at 3.13.

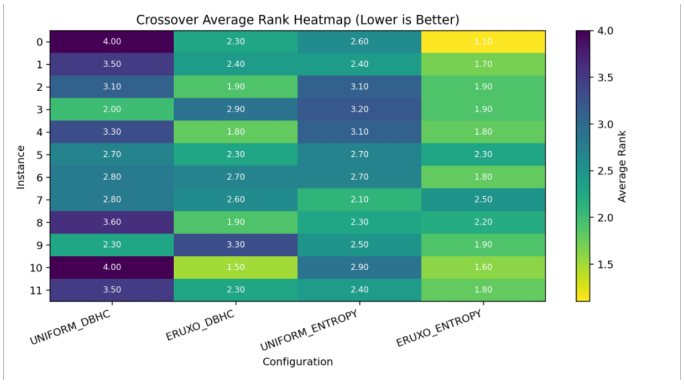


Figure 16: Per-instance average rank of crossover configurations across the 12 MAX-SAT benchmark instances. Lower values indicate better performance.

Configuration	Overall average rank
ERUXO_ENTROPY	1.88
ERUXO_DBHC	2.32
UNIFORM_ENTROPY	2.67
UNIFORM_DBHC	3.13

Table 4: Overall average rank of crossover configurations across the 12 MAX-SAT benchmark instances. Lower values indicate better performance.

8.5.5 Discussion

The crossover experiments provide a clear insight into the role of recombination in the present MAX-SAT framework. Standard uniform crossover is often too disruptive, especially when applied broadly throughout the run. By contrast, ER-UXO performs best under very restrictive settings, where crossover is limited to bits whose PV probabilities remain extremely close to 0.5. This indicates that crossover is most effective when used conservatively, only on genuinely uncertain positions.

The eligible-bit analysis further shows that PBIL progressively reduces the active scope of crossover as search proceeds. In this sense, the PBIL model does not merely guide crossover statically; it also determines when crossover should gradually stop being used. Overall, the results suggest that crossover in MAX-SAT is most effective when used conservatively, with PBIL uncertainty acting as a gating mechanism that limits recombination to genuinely undecided bit positions and progressively closes crossover as search stabilises.

8.6 PBIL Learning-Scope Analysis

8.6.1 Top-K PBIL Learning Scope Study

To examine how much of the population the PBIL model should learn from, a Top-K ablation study was conducted. The only factor varied was the learning scope of the probability-vector (PV) update: after replacement, the PV was updated using the best K individuals in the population, with

$$K \in \{1, 3, 5, 10, 25\}.$$

Here, $K = 1$ corresponds to the elite-only baseline, while $K = 25$ represents learning from half of the population when the population size is 50.

The experiment was evaluated in two stages. First, a representative-instance analysis was carried out to examine convergence behaviour in detail. Second, a ranking study was performed across all 12 MAX-SAT benchmark instances in order to assess overall robustness. The dataset-wide results are summarised in Figures 17 and 18.

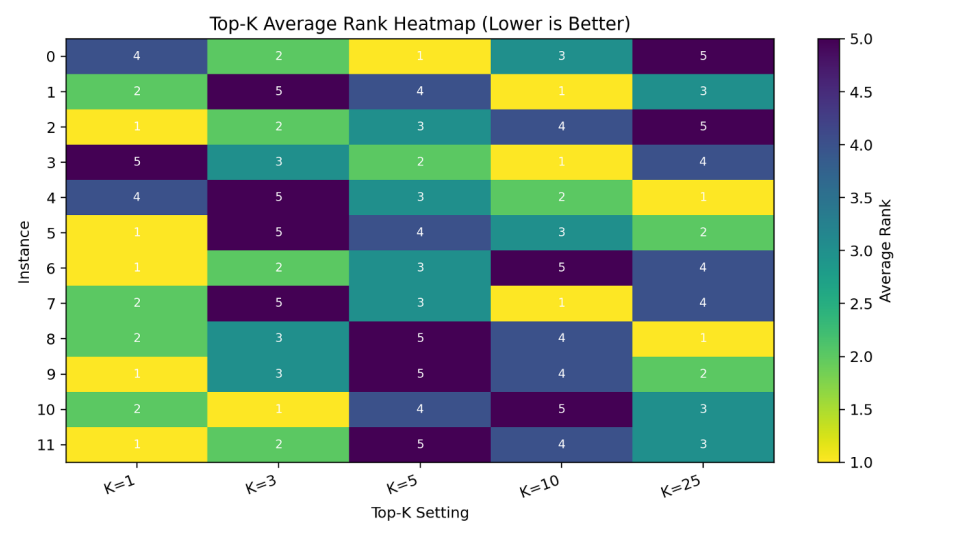


Figure 17: Instance-wise ranking of Top-K PBIL update settings across the 12 MAX-SAT benchmark instances. Lower ranks indicate better performance.

Figure 17 shows the instance-wise ranking heatmap for the five Top-K settings. Although broader learning scopes perform well on some individual instances, $K = 1$ is the most consistent setting overall. This is confirmed in Figure 18, where $K = 1$ achieves the lowest average rank across the 12-instance set, while all larger K values perform worse. Among the broader settings, $K = 5$ gives the weakest overall result.

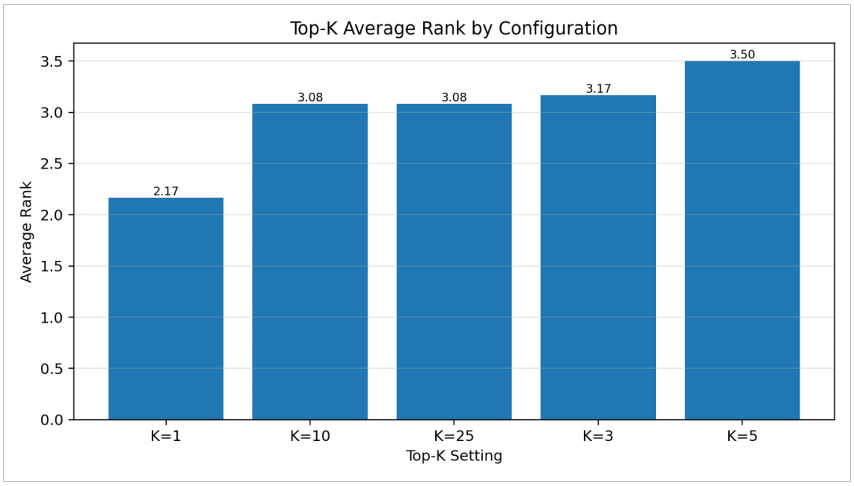


Figure 18: Average rank of each Top-K PBIL update setting across the 12 MAX-SAT benchmark instances. Lower values indicate better overall performance.

The main implication is that, in the present framework, PBIL performs best when its learning target is tightly focused on the single best solution rather than averaged across several good individuals. This keeps the PV strongly aligned with the survival pressure of the evolutionary search. By contrast, increasing K weakens the update signal by blending multiple solutions, which can dilute the guidance later provided to the PBIL-guided operators.

The results indicate that PBIL is most effective when updated toward the single best individual, suggesting that the probability vector should act as a focused intensification mechanism rather than a broad population summary. For this reason, the elite-only update ($K = 1$) was retained in the final framework design.

8.7 Combined PBIL-Guided Framework on MAX-SAT

8.7.1 Paired-Run Comparison with the Baseline

After analysing local search, crossover, and PBIL update behaviour separately, the strongest design choices were combined into a final proposed MAX-SAT configuration. This proposed variant is compared directly against the baseline memetic framework:

- **Baseline**: DBHC local search + standard uniform crossover
- **Proposed**: Entropy-guided PBIL local search + entropy-restricted uniform crossover

To evaluate the comparison more directly, a paired-run ranking procedure was used across the 12 MAX-SAT benchmark instances. For each instance, the baseline and proposed methods were run 31 times under the same experimental budget. In each paired run, the better-performing method was assigned rank 1 and the worse-performing method rank 2, with ties assigned rank 1.5. These ranks were then averaged across runs to measure how consistently one method outperformed the other. Unlike instance-level aggregation based on a single summary statistic, paired-run ranking retains run-level consistency information and therefore gives a clearer view of how reliably one configuration outperforms the other.

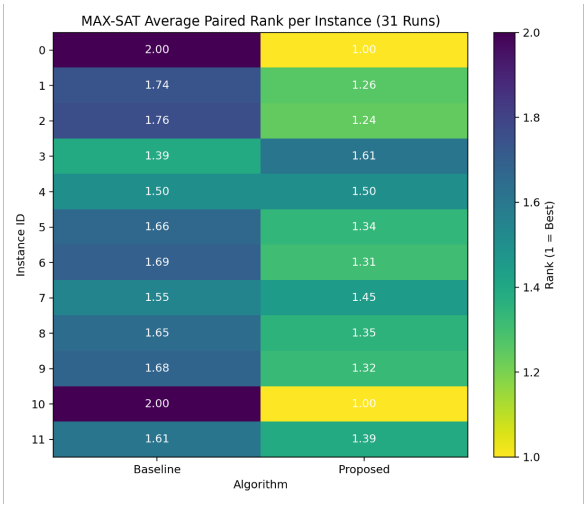


Figure 19: Average paired rank per MAX-SAT instance over 31 runs for the baseline and proposed configurations. Lower values indicate better performance.

Algorithm	Overall average paired rank
Proposed	1.31
Baseline	1.69

Table 5: Overall average paired rank across the 12 MAX-SAT benchmark instances for the baseline and proposed configurations. Lower values indicate better performance.

Figure 19 shows the average paired rank for each method on each MAX-SAT instance. The proposed configuration achieves the better average rank on most instances, indicating that its advantage is not limited to a small number of isolated runs. In several cases, the gap is substantial, while on a smaller number of instances the difference is weaker or close to neutral. This supports the view that the proposed configuration improves robustness as well as final performance.

This trend is summarised more clearly in Table 5, which reports the overall average paired rank across all 12 instances. The proposed method obtains the lower overall rank, demonstrating that the combined PBIL-guided design outperforms the baseline more consistently over repeated runs. Since the ranking is computed directly from run-level pairings rather than from a single summary statistic per instance, this result provides stronger evidence that the improvement is stable rather than incidental.

Overall, the combined results indicate that the full proposed MAX-SAT framework provides a meaningful improvement over the baseline memetic algorithm. The gain appears to come from the interaction of the two guided mechanisms: entropy-guided PBIL local search improves variable selection during refinement, while entropy-restricted crossover limits recombination to genuinely uncertain positions and becomes progressively less disruptive as PBIL confidence increases.

The paired-run analysis shows that the proposed configuration does not merely improve average performance, but outperforms the baseline more consistently across repeated runs on the MAX-SAT benchmark set.

8.8 Cross-Domain Generality on Knapsack

After analysing the proposed PBIL-guided memetic framework on MAX-SAT, the next step is to evaluate whether the same optimisation principles remain effective on a different binary combinatorial problem. For this purpose, the framework is transferred to the 0–1 Knapsack problem as a cross-domain generality test. The aim is not to redesign the algorithm for a new domain, but to examine whether the same PBIL-guided search structure can still provide useful guidance when the problem characteristics change. This is consistent with the overall goal of the project, which is to develop a unified binary optimisation framework in which the probability vector supports operator behaviour across domains rather than only within a single problem setting.

The knapsack domain provides a meaningful transfer test because it shares the same binary representation as MAX-SAT, while introducing different search characteristics through feasibility constraints and profit-weight trade-offs. This makes it possible to evaluate whether the benefits of PBIL guidance are genuinely general, or whether they depend too strongly on the structural properties of MAX-SAT. In this sense, the knapsack experiments are used to assess the robustness of the proposed framework under a consistent high-level optimisation structure.

8.8.1 Aim and Experimental Setting

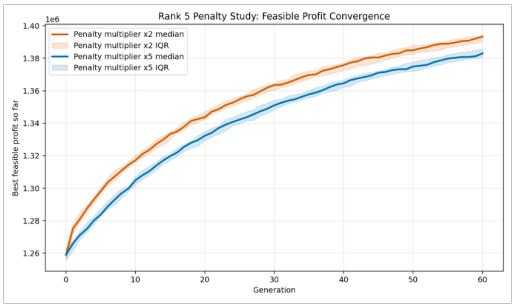
The aim of this section is to test whether the proposed PBIL-guided memetic framework can generalise beyond MAX-SAT and remain effective on the 0–1 Knapsack problem. More specifically, the experiment examines whether the same local-search guidance ideas developed earlier, particularly entropy-based guidance, continue to provide useful search control in a different binary optimisation domain. The evaluation therefore focuses on transferability of the framework rather than on domain-specific redesign.

To ensure a fair cross-domain comparison, the knapsack experiments use the same overall PBIL-guided memetic pipeline as in MAX-SAT. The core algorithmic structure is unchanged: tournament selection, crossover, mutation, local search, replacement, and PBIL probability-vector update are all retained within the same iterative workflow. The main global parameter settings are also kept fixed across domains, including a population size of 50, 60 generations, 31 independent runs, and PBIL learning rate ($\alpha = 0.01$). This allows any performance differences to be interpreted mainly in terms of domain characteristics and operator behaviour, rather than changes to the optimisation framework itself.

The knapsack benchmark set contains 12 instances drawn from the Pisinger 0–1 Knapsack dataset. These instances cover four standard correlation classes: uncorrelated, weakly correlated, strongly correlated, and inverse strongly correlated. For each class, instances are selected from different size bands so that the evaluation includes variation in both structural properties and problem scale. This benchmark design provides a suitable basis for testing whether the PBIL-guided mechanisms remain effective under different item relationships and instance sizes.

8.8.2 Penalty Multiplier Selection

To select the penalty multiplier m in $\lambda = m \times \lambda_{\text{auto}}$, a focused comparison between $m = 2$ and $m = 5$ was conducted on the Rank 5 Pisinger instance (knapPI_2_1000_10000_25) using the PBIL_MA_DBHC_IE configuration over 31 independent runs. As shown in Figure 20, the 2× multiplier consistently yields higher feasible profit, achieving a median of 1,393,330 versus 1,383,004 for 5×, with a tighter distribution. Consequently, $\lambda = 2 \times \lambda_{\text{auto}}$ was adopted for all knapsack experiments.



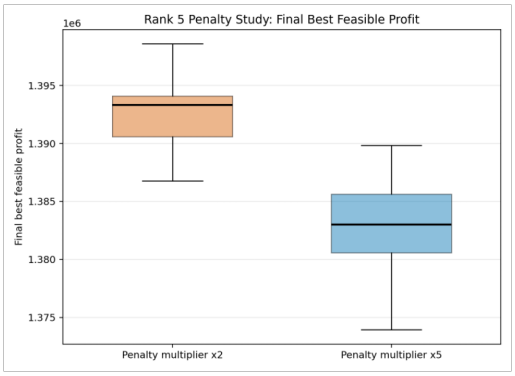


Figure 20: Penalty multiplier comparison on the Rank 5 instance. The 2× multiplier consistently yields higher feasible profit than 5×.

8.8.3 Local Search Performance on Knapsack

This subsection evaluates whether the local-search mechanisms developed for MAX-SAT remain effective after transfer to the 0–1 Knapsack problem. Four variants are compared under the same PBIL-guided memetic framework: the baseline DBHC, entropy-guided DBHC, direct PBIL-guided DBHC, and SDHC.

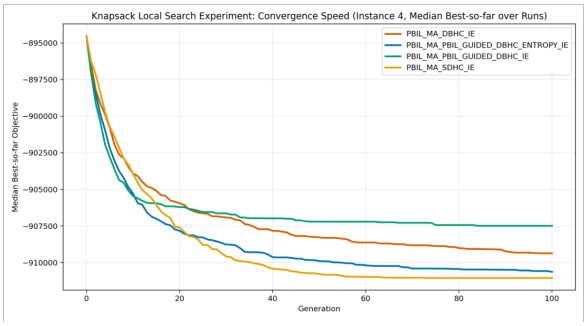


Figure 21: Knapsack local-search convergence on representative Instance 4.

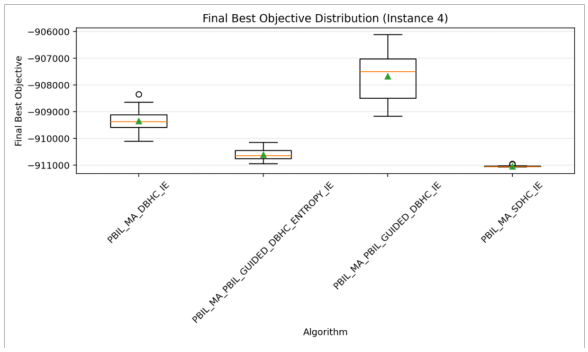


Figure 22: Final best-objective distribution on representative Instance 4 over 31 runs.

To understand the dynamic behaviour and run-level stability of these methods, performance was first analysed in detail on a representative problem, Instance 4. The convergence curves in Figure 21 show that all methods improve quickly in early generations, but their later behaviour separates them clearly. On the representative instance, SDHC achieves the strongest final convergence, while the entropy-guided variant also improves steadily and finishes ahead of the baseline DBHC and the direct PBIL-guided variant.

This pattern is supported by the final best-objective distributions shown in Figure 22. SDHC yields the best final distribution on Instance 4, and entropy-guided DBHC remains relatively stable in second place. Consistent with the convergence data, baseline DBHC performs worse than entropy-guided DBHC on this instance, while direct PBIL-guided DBHC is clearly the weakest. However, a representative instance does not necessarily reflect the overall benchmark trend, so a dataset-level comparison is required.

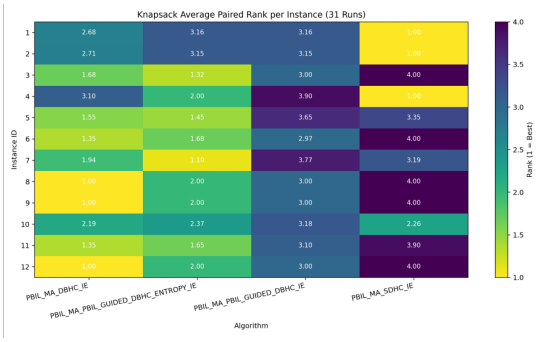


Figure 23: Average paired rank per algorithm across the 12 knapsack instances.

Algorithm	Overall average paired rank
MA_DBHC_IE	1.80
MA_PBIL_Guided_DBHC_Entropy_IE	1.99
MA_SDHC_IE	2.98
MA_PBIL_Guided_DBHC_IE	3.24

Table 6: Overall average paired rank across the 12 knapsack instances. Lower values indicate better overall performance.

To assess whether these observations generalise, a paired-run ranking procedure was applied across the full 12-instance benchmark set. Figure 23 displays the average paired rank per algorithm across all instances. The heatmap reveals a different pattern from the single-instance view: DBHC is best on most instances and exhibits the strongest overall ranking pattern, while entropy-guided DBHC is usually second and often performs closely to DBHC. In contrast, SDHC is highly inconsistent—very strong on a few instances, but very poor on many others. Direct PBIL-guided DBHC is comprehensively weak across the dataset.

This explains the apparent contradiction of SDHC's strong performance on Instance 4. As summarised by the overall average paired rank in Table 6, DBHC achieves the best overall average rank, with entropy-guided DBHC following as a close second. Both SDHC and direct PBIL-guided DBHC are clearly worse overall. Therefore, the most successful transfer on knapsack is not direct model-following guidance, but uncertainty-based guidance.

The knapsack results support partial cross-domain transfer of the proposed framework. In particular, entropy-guided local search remains competitive and transfers more successfully than direct PBIL-guided ordering, although the baseline DBHC is the strongest overall method in this constrained domain. This likely occurs because the probability vector is less reliable as a direct search guide under constrained, penalty-based search compared to the unconstrained MAX-SAT environment.

8.8.4 Crossover Transfer on Knapsack

This subsection tests whether the proposed entropy-restricted uniform crossover (ER-UXO) also transfers effectively to the knapsack domain. As in the MAX-SAT study, the crossover analysis is conducted in two stages. First, the uncertainty threshold τ is tuned on a representative instance in order to identify a reasonable restriction level for crossover. Second, the selected setting is evaluated across the full 12-instance benchmark set to determine whether the same crossover mechanism improves overall robustness.

Both methods use the same overall configuration: a population size of 50, a tournament size of 3, a mutation rate of $1/n$, DBHC local search, elitist replacement, a PBIL learning rate of 0.01, an update-top-$K = 1$ rule, feasible-first elite selection, 60 generations, and 31 runs. The only difference lies in the crossover variant:

- **Baseline:** Standard Uniform Crossover (UniformXO)
- **Proposed:** ER-UXO with $\tau = 0.3$

Standard UniformXO swaps differing bits with a fixed probability of 0.5. In contrast, ER-UXO only swaps bits when the PBIL model is still uncertain. In the knapsack setting, this restriction must be tuned more cautiously because feasibility pressure already narrows the search dynamics.

Figure 24 shows the fine-tuning result on representative Instance 6. Compared with the MAX-SAT crossover study, the pattern here is much weaker: no threshold produces a clear improvement over the baseline, and the differences between settings remain small. Nevertheless, $\tau = 0.3$ was retained as the most reasonable setting for the benchmark-wide comparison because it provides a moderate restriction level without collapsing crossover to an almost inactive mechanism.

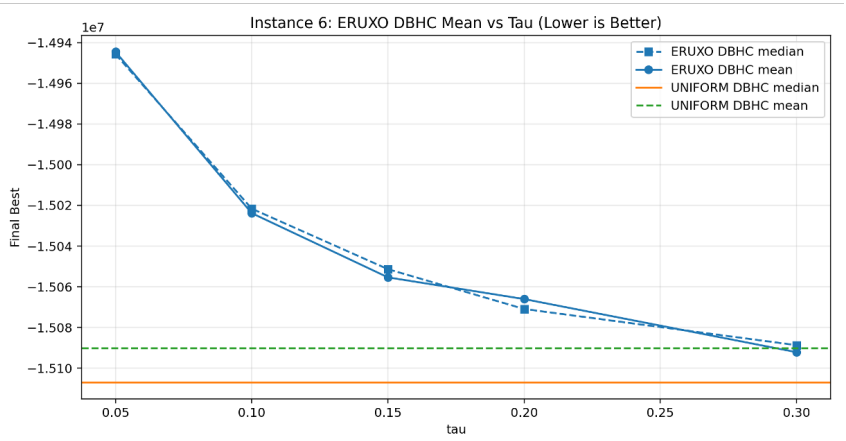


Figure 24: Mean and median final best objective value on Knapsack Instance 6 under different ER-UXO uncertainty thresholds (τ), compared with the standard uniform-crossover baseline. Lower values indicate better performance.

After fixing $\tau = 0.3$, the crossover comparison was extended to all 12 knapsack instances using the same paired-run ranking procedure as in the earlier analyses.

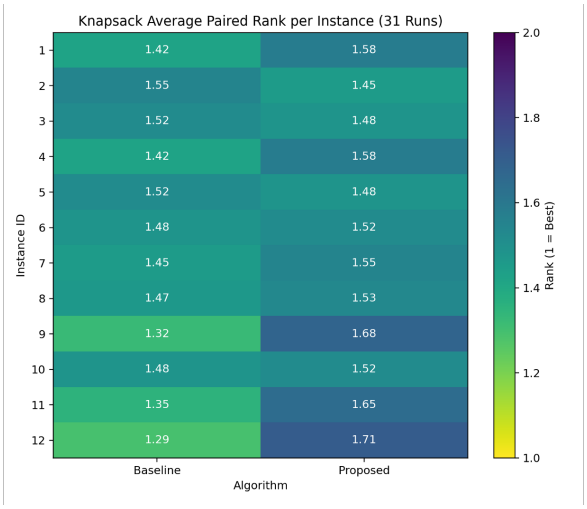


Figure 25: Average paired rank per crossover algorithm across the 12 knapsack instances.

Algorithm	**Overall average paired rank**
Baseline	1.439
Proposed	1.561

Table 7: Overall average paired rank for crossover variants across all 12 knapsack instances. Lower values indicate better overall performance.

Figure 25 shows the per-instance paired-rank heatmap, while Table 7 reports the overall average paired rank. The baseline standard UniformXO performs better on a majority of the instances. Although ER-UXO remains competitive in a few cases, it is not consistently superior, and the corrected overall ranking confirms that the baseline is better on average (1.439 vs 1.561).

These results indicate that ER-UXO does not outperform standard UniformXO on the knapsack domain under the current setting. Unlike MAX-SAT, the uncertainty threshold does not provide a clear benefit here and instead appears to have a slightly negative overall effect. A plausible explanation is that knapsack search is already strongly shaped by feasibility pressure, so further restricting crossover can reduce useful diversity, accelerate convergence, and cause the search to commit too early to misleading PBIL beliefs.

Overall, the knapsack crossover study shows that entropy-restricted crossover does not transfer as successfully as it did in MAX-SAT. The τ tuning on Instance 6 suggests only weak sensitivity, and the full benchmark comparison shows that the restricted crossover is slightly worse overall. This indicates that, in the knapsack domain, PBIL-based crossover gating may close exploration too early rather than improving recombination quality.

8.9 Shared-Evaluation Comparison on MAX-SAT

All preceding MAX-SAT experiments in this evaluation used a fixed number of generations as the stopping condition. Under that design, memetic algorithms with local search consume substantially more fitness evaluations per generation than algorithms without local search, because every local-search step requires additional objective-function evaluations. This means that a fixed-generation comparison implicitly grants a larger computational budget to methods that perform local search, making it difficult to determine whether their superior performance comes from better search guidance or simply from additional computation. To address this, a shared-evaluation comparison was conducted in which all five algorithm variants receive the same total number of fitness evaluations, regardless of how those evaluations are distributed across generations and local-search steps.

Five algorithms were compared: **GA** (genetic algorithm with no local search), **PBIL** (standalone probability-vector learning with no crossover or local search), **MA-DBHC** (memetic algorithm with Davis's Bit Hill Climbing), **MA-SDHC** (memetic algorithm with Steepest Descent Hill Climbing), and **PBIL-MA-Entropy** (the proposed framework with entropy-guided DBHC local search and entropy-restricted crossover). All algorithms were evaluated on the same 12 MAX-SAT benchmark instances listed in Table 1, using 31 independent runs per algorithm. The shared parameters were: population size 50, tournament size 3, bit-flip mutation with rate $1/n$, and elitist transgenerational replacement. The shared evaluation budget was set to 755,678 fitness evaluations, determined by running PBIL-MA-Entropy on Instance 6 for 60 generations across 31 runs and taking the median total evaluation count. Since MAX-SAT is treated as a minimisation problem in this framework, lower values of `final_best` (the number of unsatisfied clauses) indicate better performance.

Instance	GA	PBIL	MA-DBHC	MA-SDHC	PBIL-MA-Entropy
0	**31**	48	80	245	32
1	39	53	33	222	**26**
2	33	47	26	215	**21**
3	27	34	12	111	**9**
4	40	42	23	113	**13**
5	54	59	47	216	**23**
6	13	21	**8**	21	**8**
7	13	21	**7**	21	**7**
8	16	27	11	31	**10**
9	225	249	216	265	**215**
10	31	54	57	210	**26**
11	16	27	11	31	**10**

Table 8: Median final best objective value (unsatisfied clauses) across 31 runs for each algorithm on the 12 MAX-SAT benchmark instances under a shared evaluation budget of 755,678. Lower values indicate better performance. Bold entries mark the best result per instance.

Algorithm	Average rank	Instance wins
PBIL-MA-Entropy	1.167	11
MA-DBHC	2.250	2
GA	2.750	1
PBIL	3.917	0
MA-SDHC	4.917	0

Table 9: Overall average rank and number of instance wins for each algorithm across the 12 MAX-SAT instances under shared evaluation budget. Lower average rank indicates better overall performance.

Table 8 reports the median final best objective value for each algorithm on every instance, and Table 9 summarises the overall ranking. PBIL-MA-Entropy achieves the best or tied-best median on 11 of the 12 instances and obtains the strongest overall average rank of 1.167. The only instance where it does not win outright is Instance 0, where GA achieves the best median (31 versus 32). On Instances 6 and 7, MA-DBHC ties with PBIL-MA-Entropy. MA-DBHC is a clear second overall (average rank 2.250), confirming that DBHC-based local search is effective on MAX-SAT even without PBIL guidance.

Several results are noteworthy. GA, despite having no local search, achieves a respectable third place with an average rank of 2.750 and wins one instance. Under a shared evaluation budget, GA can invest all evaluations into population-level recombination, which partly compensates for the absence of local refinement. MA-SDHC, by contrast, performs very poorly with an average rank of 4.917, consistently finishing last or near-last. Its steepest-descent approach consumes many evaluations per generation for limited benefit on MAX-SAT, consistent with the earlier local-search findings (Table 3). Standalone PBIL is also weak (average rank 3.917), confirming that PBIL's value in this framework is as a guidance mechanism for operators rather than as an independent optimiser.

These results strengthen the earlier findings from the fixed-generation experiments. The proposed PBIL-MA-Entropy framework is the strongest method on MAX-SAT, and crucially this dominance holds even when the comparison is made under a shared evaluation budget that removes the implicit advantage of local-search-based methods receiving more fitness evaluations. Combined with the earlier paired-run analysis (Table 5), this provides the strongest evidence in the evaluation that entropy-guided PBIL search genuinely improves solution quality rather than merely benefiting from additional computational effort.

8.10 Shared-Evaluation Comparison on Knapsack

The same shared-evaluation methodology was applied to the 0–1 Knapsack domain in order to test whether the cross-domain findings from the earlier knapsack experiments hold under a fair evaluation budget. This experiment determines which algorithm variant performs best on knapsack when all methods receive the same total number of fitness evaluations, providing a definitive cross-domain comparison.

This experiment uses a different set of knapsack instances from the main benchmark listed in Table 2. Whereas the main benchmark covers Pisinger classes C1–C4 with item counts $n \in \{200, 1000, 5000, 10000\}$, the shared-evaluation experiment uses classes C1–C6 with item counts $n \in \{50, 500\}$. The broader class coverage includes two additional correlation structures: class C5 (almost-strongly correlated, where profit $\approx$ weight + constant with slight noise) and class C6 (subset-sum, where profit = weight). The smaller item counts ensure that all five algorithms, including those without local search, can make meaningful progress within the shared budget. This instance set complements the main benchmark by providing additional evidence on algorithm behaviour across a wider range of knapsack structures.

ID	Class	Class description	Items (n)
1	1	Uncorrelated	50
2	1	Uncorrelated	500
3	2	Weakly correlated	50
4	2	Weakly correlated	500
5	3	Strongly correlated	50
6	3	Strongly correlated	500
7	4	Inverse strongly correlated	50
8	4	Inverse strongly correlated	500
9	5	Almost-strongly correlated	50
10	5	Almost-strongly correlated	500
11	6	Subset sum	50
12	6	Subset sum	500

Table 10: Knapsack instances used in the shared-evaluation experiment. Classes are drawn from the Pisinger dataset [30], covering all six standard correlation classes at two problem sizes.

Five algorithms were compared: **GA**, **PBIL**, **MA-DBHC**, **MA-SDHC**, and **PBIL-MA**. Note that PBIL-MA in this experiment uses standard UniformXO rather than entropy-restricted crossover, consistent with the earlier finding that ER-UXO does not transfer effectively to the knapsack domain

(Table 7). The PBIL-MA variant retains entropy-guided DBHC local search and PBIL model learning. The shared evaluation budget was 4,517,996 fitness evaluations, calibrated by running PBIL-MA on Instance 6 (strongly correlated, $n = 500$) for 180 generations across 31 runs and taking the median total evaluation count. All other settings match the MAX-SAT experiment: population size 50, tournament size 3, bit-flip mutation with rate $1/n$, elitist transgenerational replacement, and 31 independent runs per algorithm. Since the knapsack problem is a maximisation problem, higher median best feasible profit indicates better performance.

ID	C	n	GA	PBIL	MA-DBHC	MA-SDHC	PBIL-MA
1	1	50	**8373**	8184	7478	**8373**	7177
2	1	500	16694	7977	8708	**22880**	8425
3	2	50	**1396**	**1396**	**1396**	**1396**	**1396**
4	2	500	**4566**	**4566**	4503	4556	4539
5	3	50	**1894**	**1894**	**1894**	**1894**	**1894**
6	3	500	**7117**	**7117**	7017	**7117**	7116
7	4	50	**994**	**994**	**994**	**994**	**994**
8	4	500	**2712**	2711	**2712**	**2712**	**2712**
9	5	50	**2096**	**2096**	**2096**	**2096**	**2096**
10	5	500	**7241**	**7241**	7232	7240	7238
11	6	50	**994**	**994**	**994**	**994**	**994**
12	6	500	**2517**	**2517**	**2517**	**2517**	**2517**

Table 11: Median best feasible profit across 31 runs for each algorithm on the 12 knapsack instances under a shared evaluation budget of 4,517,996. Higher values indicate better performance. Bold entries mark the best result per instance.

Algorithm	Average rank	Instance wins
GA	2.417	11
MA-SDHC	2.583	10
PBIL	3.000	9
PBIL-MA	3.458	7
MA-DBHC	3.542	7

Table 12: Overall average rank and number of instance wins for each algorithm across the 12 knapsack instances under shared evaluation budget. Lower average rank indicates better overall performance.

Table 11 reports the median best feasible profit per instance, and Table 12 summarises the overall ranking. GA achieves the best overall average rank (2.417), closely followed by MA-SDHC (2.583). PBIL is mid-table at 3.000, while PBIL-MA and MA-DBHC are the weakest at 3.458 and 3.542 respectively. This ranking is strikingly different from the MAX-SAT results (Table 9), where the proposed framework dominated and GA was only third.

The per-instance results reveal a clear pattern related to problem size. On the small instances ($n = 50$), most or all algorithms achieve the same median profit across many classes (C2, C3, C4, C5, C6), indicating that these small problems are effectively solved to near-optimality by all methods and do not differentiate the algorithms. The meaningful separation occurs on the medium instances ($n = 500$). Among these, Instance 2 (C1 Uncorrelated, $n = 500$) is the most dramatic: MA-SDHC achieves a median profit of 22,880, far ahead of the next-best GA at 16,694 and all other algorithms below 9,000. This suggests that steepest-descent local search is particularly effective on large uncorrelated instances where the search landscape has many local improvements available. On the remaining $n = 500$ instances, GA and PBIL

are the most consistently strong performers.

A plausible explanation for GA's strong performance on knapsack is that crossover, specifically UniformXO, is a more effective search operator in this domain than local search. GA can invest its entire evaluation budget in population-level recombination without expending evaluations on per-solution hill climbing. This interpretation is consistent with the earlier finding that standard UniformXO outperformed entropy-restricted crossover on knapsack (Table 7). Furthermore, MA-DBHC and PBIL-MA achieve nearly identical overall performance (average ranks 3.542 and 3.458 respectively), which suggests that PBIL guidance provides negligible additional benefit on knapsack beyond what standard DBHC local search already offers. The feasibility pressure imposed by the knapsack capacity constraint already provides strong structural guidance to the search, leaving less room for the PBIL model to contribute meaningfully.

Comparing across the two domains, a clear pattern emerges. On MAX-SAT, sophisticated local-search guidance is the primary driver of performance: PBIL-MA-Entropy dominates because entropy-guided DBHC focuses search effort on genuinely uncertain bit positions, and entropy-restricted crossover prevents premature recombination. On knapsack, population-level recombination is the more effective mechanism: GA dominates because UniformXO is well-suited to the knapsack structure, and local search adds comparatively less value under feasibility pressure. These shared-evaluation results provide the fairest comparison in this dissertation and confirm that the proposed framework's strengths are domain-dependent. The PBIL-guided memetic approach delivers its clearest advantage on problems where local search is the primary bottleneck for solution quality, such as MAX-SAT, but adds less value on constrained problems where recombination and feasibility management already drive performance effectively.

9 Summary & Reflection

9.1 Overall Summary of Findings

This dissertation investigated whether a PBIL probability vector can be embedded directly into the operator-level decision-making of a Memetic Algorithm to improve performance on binary combinatorial optimisation problems. A unified co-evolutionary framework was developed in which the PBIL model is continuously learned from the evolving population and used to guide both local search and crossover.

On the primary MAX-SAT domain, the proposed framework consistently outperformed the baseline memetic algorithm. Entropy-guided DBHC local search achieved the best overall paired rank across the 12-instance benchmark set (Table 3), and when combined with entropy-restricted crossover, the full configuration outperformed the baseline more consistently on a run-by-run basis (Table 5). Crucially, the shared-evaluation comparison confirmed that this advantage is not an artefact of additional fitness evaluations: under an identical budget of 755,678 evaluations, PBIL-MA-Entropy won 11 of 12 instances and achieved an average rank of 1.167 across five competing algorithms (Table 9).

On the 0–1 Knapsack domain, cross-domain transfer was only partial. Entropy-guided local search remained competitive, but entropy-restricted crossover did not reliably improve performance in this constrained domain (Table 7). The shared-evaluation comparison on knapsack sharpened this finding: under equal evaluation budgets, the standard GA was the strongest algorithm (average rank 2.417) while PBIL-MA was near last (average rank 3.458), indicating that the framework's advantage does not generalise to problems where feasibility pressure, rather than local-search guidance, is the primary performance driver (Table 12). This partial transfer is itself a valuable finding, because it delineates where the approach is robust and where it is sensitive to problem structure.

9.2 Main Contributions and Originality

The main contribution of this dissertation is not the use of PBIL as an optimiser, but the idea of embedding a PBIL probability vector directly into operator behaviour within a unified memetic framework. This establishes a co-evolutionary interaction in which the population and the probabilistic model learn from each other over generations, and the learned model actively shapes how local search and crossover operate at the bit level. This addresses the research gap identified in the earlier chapters: existing literature typically decouples PBIL-style learning from fine-grained operator decisions, using the learned model only for sampling new solutions rather than guiding operator internals.

Within this framework, four specific original contributions were made. First, entropy-guided DBHC was proposed and evaluated as a local-search strategy that uses PV uncertainty to determine bit visitation order, directing search effort toward positions where the model remains uncertain rather than toward

its current preference. Second, entropy-restricted uniform crossover was proposed and evaluated as a recombination strategy that permits bit swapping only at positions where the PBIL model has not yet developed a confident preference, progressively narrowing the scope of crossover as model confidence increases and effectively allowing the PBIL model to act as a gating mechanism for recombination. Third, the framework was analysed across several design dimensions including learning rate sensitivity, Top-K learning scope, eligible-bit dynamics, and cross-domain transfer. Fourth, a shared-evaluation comparison methodology was developed in which all five algorithm variants were compared under identical total evaluation budgets on both MAX-SAT and knapsack (Tables 9 and 12), providing a fair cross-algorithm comparison that separates genuine algorithmic advantage from budget confounds.

9.3 Reflection on the Results

The results reveal an important and perhaps counter-intuitive insight: PBIL guidance is most effective when used to identify uncertainty rather than to enforce preference. Direct PBIL-guided DBHC, which prioritises bit positions according to their conflict with the model's current preference, performed consistently worse than the baseline on both MAX-SAT (Table 3) and knapsack (Table 6). In contrast, entropy-guided local search succeeded precisely because it directed search effort toward positions where the model was least confident — the positions where productive bit flips are most likely. The shared-evaluation comparisons reinforce this point from a different angle: standalone PBIL, which uses the probability vector directly as an optimiser, achieved average ranks of only 3.917 on MAX-SAT and 3.000 on knapsack (Tables 9 and 12). PBIL's value lies in guiding operators, not in replacing them.

The shared-evaluation experiments also crystallise the domain-dependence of the framework. On MAX-SAT, PBIL-MA-Entropy dominated even under fair evaluation budgets (average rank 1.167 versus 2.750 for the standard GA), confirming that local-search guidance is the primary performance driver on unconstrained satisfaction problems. On knapsack, the picture reversed: the standard GA was strongest (average rank 2.417) and PBIL-MA was near last (average rank 3.458). This contrast arises because the capacity constraint already imposes structural guidance on the search, making PBIL's contribution largely redundant, while the feasibility pressure distorts the probability vector and renders entropy-restricted crossover counterproductive (Table 7).

Taken together, these observations suggest that lightweight probabilistic models can serve as effective operator-level signals, but only when the model reliably reflects genuine structural uncertainty in the problem. When external constraints distort the learned distribution, the model's entropy ceases to be a reliable guide, and simpler operators perform equally well or better. The framework's strength is therefore conditional: it is most valuable on problems where local search is the performance bottleneck and where the objective landscape is not distorted by feasibility constraints.

9.4 Reflection on Project Development and Plan Evolution

The development of this dissertation followed two linked planning stages. The initial Phase 1 plan focused on proposal and ethics preparation, literature review, core algorithm implementation, a preliminary study, and preparation of the interim report. This first stage was intended to establish the technical and methodological foundation of the project before the second-semester work was planned in more detail.

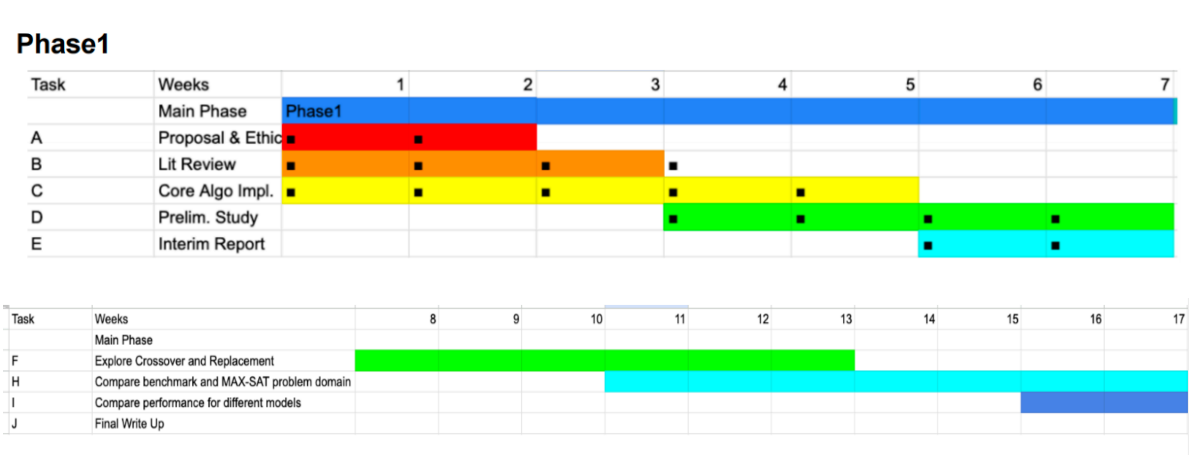


Figure 26: Original Phase 1 and interim-report Gantt charts used as planning references during the interim stage. The second chart includes Stage H, which was initially framed as a benchmark comparison around the MAX-SAT problem domain before being refined in the final dissertation.

Figure 26 shows the Phase 1 and interim-report Gantt charts from the original project planning. At the interim stage, the project was still centred on MAX-SAT, and the second-stage plan treated the later work as an extension of that line of study rather than as a fully defined cross-domain investigation.

Within that original plan, the interim report marked the point at which the core PBIL-guided memetic framework had already been established and the initial MAX-SAT evaluation pipeline was in place. The first-stage experiments had covered the main local-search variants, PBIL learning-rate sensitivity, and the early experimental workflow. The second-stage chart then extended this trajectory through crossover and replacement analysis, with Stage H labelled as comparing a benchmark with the MAX-SAT problem domain.

In the final dissertation, however, Stage H was refined rather than followed literally. Instead of treating the second domain as a general benchmark add-on, the project was repositioned as a deliberate cross-domain generality study using the 0–1 knapsack problem as the transfer domain. This change improved the coherence of the dissertation because the comparison now tested whether the same PBIL-guided operator design could transfer across two genuinely different binary combinatorial problems that share a binary representation but differ in feasibility structure and profit-to-weight trade-offs.

This evolution was therefore an improvement in research design rather than a deviation in quality. The final dissertation is more clearly organised around one central question about operator-level PBIL guidance, and the knapsack results are more interpretable precisely because the second domain was chosen and justified as a structured transfer target rather than kept as a vague benchmark extension.

9.5 Limitations and Future Work

Several limitations of this work should be acknowledged. First, only two binary problem domains were studied. Although MAX-SAT and 0–1 knapsack provide a useful structural contrast, stronger claims about cross-domain generality would require evaluation on additional binary combinatorial problems, such as graph-based optimisation, set cover, or feature selection tasks.

Second, the PBIL model used throughout this work is bit-independent: each probability is learned and updated separately without modelling correlations between bit positions. Richer probabilistic models, such as Bayesian network EDAs or models that capture pairwise variable interactions, could potentially provide more informative guidance to operators, particularly on problems with strong inter-variable dependencies.

Third, as discussed in the results, entropy-restricted crossover did not transfer successfully to the knapsack domain. This suggests that the current crossover gating mechanism may need to be adapted for constrained problems, for example by incorporating feasibility-aware PV learning or by dynamically adjusting the uncertainty threshold based on the degree of constraint activity during search.

Fourth, while the framework is unified in structure, the results show that domain characteristics still strongly influence which components are beneficial. Future work could explore adaptive mechanisms that detect when PV guidance is reliable and adjust operator behaviour accordingly, rather than applying a fixed operator configuration across all domains.

9.6 Broader Reflection

From a legal, social, ethical, and professional perspective, the most significant issues in this dissertation concern professionalism in experimental design, reproducibility, intellectual-property awareness, and the responsible interpretation of optimisation results. Because the work is centred on algorithm design and benchmark evaluation rather than human participants or personal data, research-ethics and data-protection considerations were not major practical risks in the conduct of the project. A much more significant issue was fairness of algorithm comparison. Since memetic algorithms may consume substantially more objective-function evaluations than simpler baselines, it would be easy to draw misleading conclusions if methods were given unequal computational opportunities. The shared-evaluation comparisons were introduced for precisely this reason, making fair benchmarking not only a methodological choice but also an ethical and professional obligation: claims of superiority should not depend on hidden budget advantages.

Reproducibility was another central professional issue. Since optimisation results can be sensitive to randomness, parameter settings, and stopping conditions, the use of repeated independent runs, fixed high-level parameter settings, and explicit reporting of rankings and shared-budget tests all contributed to transparent, inspectable evidence. From a legal and academic-integrity perspective, the main concern was intellectual property and attribution: distinguishing clearly between standard methods taken from the literature and the original contributions of this dissertation, particularly since the project builds on established algorithms and benchmark datasets rather than inventing PBIL or memetic search independently.

The broader social significance of the project is indirect. Optimisation methods of this kind may support practical applications in scheduling, allocation, logistics, and network design, but they are tools whose consequences depend on the context in which they are deployed. A stronger optimiser could improve beneficial systems or be applied to settings where the objective being optimised is itself narrow or potentially harmful; responsible use therefore depends on the values and constraints of the application domain. A related consideration is computational cost: although each individual run is modest, the aggregate workload across many algorithm variants, repeated runs, and two problem domains is non-trivial, creating a minor but relevant environmental and professional reason to design experiments purposefully rather than run them excessively.

Overall, the most important LSEPI issues for this dissertation were fairness of evidence, reproducibility, correct attribution, and responsible interpretation of algorithmic benefit. The project does not raise major privacy or human-subject concerns, but it does require careful attention to how scientific claims are produced, justified, and communicated.

10 Declaration of AI Usage

I confirm that this dissertation is my own original work. I have not used Artificial Intelligence (AI) tools to generate academic content or ideas. Artificial intelligence tools, specifically ChatGPT (OpenAI), Gemini (Google), and Grammarly, were used in a limited supporting role throughout the dissertation for proofreading and language refinement, including grammar, spelling, and vocabulary. The same tools were also used in a limited technical-support role for debugging, fixing errors, refactoring scripts used in the experimental evaluation for the local-search and crossover studies, and adjusting some LaTeX table formatting. They were not used to generate the core academic content, research ideas, algorithmic design, main implementation, experimental results, or overall analysis of this dissertation. All underlying ideas, implementation decisions, experiments, interpretations, and conclusions are my own.

References

[1] Qasem Al-Tashi, Said Jadid Abdul Kadir, Helmi Md Rais, Seyedali Mirjalili, and Hitham Alhussian. Binary optimization using hybrid grey wolf optimization for feature selection. *IEEE Access*, 7:39496–39508, 2019.

[2] Shumeet Baluja. Population-based incremental learning: A method for integrating genetic search based function optimization and competitive learning. Technical Report CMU-CS-94-163, Carnegie Mellon University, Pittsburgh, 1994.

[3] Shumeet Baluja and Scott Davies. Using optimal dependency-trees for combinatorial optimization: Learning the structure of the search space. In *Proceedings of the Fourteenth International Conference on Machine Learning*, pages 30–38, 1997.

[4] Zahra Beheshti. UTF: Upgrade transfer function for binary meta-heuristic algorithms. *Applied Soft Computing*, 106:107346, 2021.

[5] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[6] Dalila Boughaci and Habiba Drias. Efficient and experimental meta-heuristics for MAX-SAT problems. In Sotiris E. Nikoletseas, editor, *Experimental and Efficient Algorithms*, volume 3503 of *Lecture Notes in Computer Science*, pages 501–512. Springer, Berlin, Heidelberg, 2005.

[7] Kalyanmoy Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multi-objective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[8] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, 1979.

[9] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989.

[10] Georges R. Harik, Fernando G. Lobo, and David E. Goldberg. The compact genetic algorithm. *IEEE Transactions on Evolutionary Computation*, 3(4):287–297, 1999.

[11] Mark Hauschild and Martin Pelikan. An introduction and survey of estimation of distribution algorithms. *Swarm and Evolutionary Computation*, 1(3):111–128, 2011.

[12] Federico Heras, Javier Larrosa, and Albert Oliveras. MiniMaxSAT: An efficient weighted Max-SAT solver. *Journal of Artificial Intelligence Research*, 31:1–32, 2008.

[13] John H. Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. University of Michigan Press, Ann Arbor, 1975.

[14] Matthew Hyde and Gabriela Ochoa. Hyflex competition instance summary. ASAP Group, University of Nottingham, 2011. Available at: https://people.cs.nott.ac.uk/pszwj1/chesc2011/reports/CHeSCInstanceSummary.pdf. Accessed 31 March 2026.

[15] James Kennedy and Russell C. Eberhart. A discrete binary version of the particle swarm algorithm. In *Proceedings of the 1997 IEEE International Conference on Systems, Man, and Cybernetics: Computational Cybernetics and Simulation*, volume 5, pages 4104–4108, 1997.

[16] N. Krasnogor and J. Smith. A tutorial for competent memetic algorithms: model, taxonomy, and design issues. *IEEE Transactions on Evolutionary Computation*, 9(5):474–488, 2005.

[17] Pedro Larrañaga and José A. Lozano. *Estimation of Distribution Algorithms: A New Tool for Evolutionary Computation*. Kluwer Academic Publishers, Boston, 2001.

[18] Mengjun Li, Qifang Luo, and Yongquan Zhou. BGOA-TVG: Binary grasshopper optimization algorithm with time-varying gaussian transfer functions for feature selection. *Biomimetics*, 9(3):187, 2024.

[19] Bernhard Lienland and Li Zeng. A review and comparison of genetic algorithms for the 0–1 multidimensional knapsack problem. *International Journal of Operations Research and Information Systems*, 6(2):21–31, 2015.

[20] Cláudio F. Lima, Kumara Sastry, David E. Goldberg, and Fernando G. Lobo. Combining competent crossover and mutation operators: A probabilistic model building approach. In *Proceedings of the 2005 Conference on Genetic and Evolutionary Computation*, pages 735–742. ACM, 2005.

[21] Silvano Martello, David Pisinger, and Paolo Toth. New trends in exact algorithms for the 0–1 knapsack problem. *European Journal of Operational Research*, 123(2):325–332, 2000.

[22] Silvano Martello and Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, Chichester, 1990.

[23] Ruben Martins, Vasco Manquinho, and Inês Lynce. Open-WBO: A modular MaxSAT solver. In Carsten Sinz and Uwe Egly, editors, *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 438–445. Springer, Cham, 2014.

[24] Pablo Moscato. On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms. Technical Report 826, California Institute of Technology, Pasadena, 1989.

[25] Pablo Moscato and Carlos Cotta. A gentle introduction to memetic algorithms. In Fred Glover and Gary A. Kochenberger, editors, *Handbook of Metaheuristics*, pages 105–144. Kluwer Academic Publishers, Boston, MA, 2003.

[26] H. Mühlenbein and G. Paaß. From recombination of genes to the estimation of distributions I. Binary parameters. In H.-M. Voigt, W. Ebeling, I. Rechenberg, and H.-P. Schwefel, editors, *Parallel Problem Solving from Nature — PPSN IV*, pages 178–187. Springer, Berlin and Heidelberg, 1996.

[27] Ender Özcan and Can Başaran. A case study of memetic algorithms for constraint optimization. *Soft Computing*, 13(8–9):871–882, 2009.

[28] Jeng-Shyang Pan, Pei Hu, Václav Snášel, and Shu-Chuan Chu. A survey on binary metaheuristic algorithms and their engineering applications. *Artificial Intelligence Review*, 56(7):6101–6167, 2023.

[29] M. Pelikan, D. E. Goldberg, and F. G. Lobo. A survey of optimization by building and using probabilistic models. *Computational Optimization and Applications*, 21(1):5–20, 2002.

[30] David Pisinger. Where are the hard knapsack problems? *Computers & Operations Research*, 32(9):2271–2284, 2005.

[31] Kate Smith-Miles, Jeffrey Christiansen, and Mario Andrés Muñoz. Revisiting "where are the hard knapsack problems?" via instance space analysis. *Computers & Operations Research*, 128:105184, 2021.

[32] El-Ghazali Talbi. A taxonomy of hybrid metaheuristics. *Journal of Heuristics*, 8:541–564, 2002.

[33] Zequn Wei and Jin-Kao Hao. A threshold search based memetic algorithm for the disjunctively constrained knapsack problem. *Computers & Operations Research*, 136:105447, 2021.

[34] Shengxiang Yang. Memory-enhanced univariate marginal distribution algorithms for dynamic optimization problems. In *Proceedings of the 2005 IEEE Congress on Evolutionary Computation*, volume 3, pages 2560–2567. IEEE Press, 2005.

[35] Shengxiang Yang and Xin Yao. Experimental study on population-based incremental learning algorithms for dynamic optimization problems. *Soft Computing*, 9(11):815–834, 2005.